LAHORE GARRISON UNIVERSITY

Department of Computer Science

BS Computer Science — Fall 2026

Semester Capstone Project Specification

AI-Native Web-Based Operating System (WebOS)

An operating system that boots, schedules, pages, persists and reasons entirely inside a web browser

---

Disciplines integrated	Operating Systems; Data Structures & Algorithms; Database Systems; Artificial Intelligence
Team size	Exactly five students per team
Duration	18 weeks, delivered in five phases
Programme	BS Computer Science
Issued by	Mr. Haseeb Ur Rehman, Course Coordinator
Document version	Version 1.0 — September 2026

About This Document

This is the complete and authoritative specification for the Fall 2026 semester capstone project. It explains why the project exists and what your team must build. It gives the measurable acceptance criteria your system will be tested against, the phases in which it will be delivered, and how it will be graded.

If this document disagrees with an informal message, a slide or a conversation, this document prevails. It changes only through a numbered written amendment posted on Google Classroom. Read Part I together as a team before you choose a stack, a name or a role.

The project at a glance

What you build	A named, bootable, AI-native operating system that runs inside a modern browser and is backed by a multi-user server.
Team	Exactly five students per team, each owning a defined engineering role (Section 4).
Disciplines integrated	Operating Systems, Data Structures & Algorithms, Database Systems and Artificial Intelligence. Each has mandatory, testable requirements.
Identity	Your team names its operating system and builds its own boot screen, boot sequence and login screen (Sections 5 and 6).
Technology	Open stack: any language, framework, runtime or database. Every choice must be defended in writing (Section 14).
AI mandate	At least two of three system-level AI subsystems. AI must be part of the OS, not a chatbot bolted on top of it (Section 12).
Delivery	Five phases, from the architecture defence (Week 4) to the final defence (Weeks 17–18), each closed by a live gate review (Section 13).
Assessment	100 marks across six criteria. Team marks are moderated for each member using the viva and contribution evidence (Section 16).

How to read this document

Part	Contents	Who must read it closely
I — Why	Why web operating systems matter now, what separates a real OS from an imitation, and why AI must be a foundation of the system.	Everyone, before the first team meeting.
II — The Brief	Team structure, naming your OS, the mandatory boot screen and boot sequence, and the industry benchmarks you must surpass.	Everyone.
III — Specification	Reference architecture, then one section per subsystem: OS primitives, DSA, database, AI.	Each owner reads their section closely; everyone reads Section 8.
IV — Delivery	Milestones, the architecture defence, repository and testing standards, the rubric, the final defence and integrity rules.	Everyone, and again before every gate.

Part	Contents	Who must read it closely
Appendices	Acceptance checklist, glossary, references and the team registration form.	QA/DevOps lead, before every gate.

Conventions

Requirement keywords. The words **must**, **should** and **may** follow their usual meaning in engineering specifications (RFC 2119). *Must* is mandatory for passing a gate and for full marks. *Should* is expected unless you justify a different choice in writing. *May* is optional and earns credit when done well.

Requirement identifiers. Every testable requirement has an identifier. Use these identifiers in your test names, pull-request titles and report, for example a test named `VFS-06 rejects a write that exceeds the quota without partial data`.

Prefix	Area	Section	Prefix	Area	Section
BOOT	Boot screen & sequence	6	**SYS**	System calls & terminal	9.6
PROC	Processes & scheduling	9.1	**DSA**	Data structures	10
MEM	Virtual memory & paging	9.2	**DB**	Database & integrity	11
VFS	Virtual file system	9.3	**RAG**	Semantic VFS search	12.2
WM	Window management	9.4	**AGT**	Agentic system execution	12.3
OFF	Offline execution & sync	9.5	**PRE**	Predictive pre-fetching	12.4

Measurement. Numerical targets are measured on a standard laboratory workstation in the Department, whose specification is published in Week 2, using the current stable Google Chrome at the time of each gate.
Your OS name. This document suggests no names. Wherever it shows *YourOS*, your team's own registered name goes.

Contents

Part I: Why This Project Exists

1 Executive Summary 6
1.1 What your team will deliver 6
1.2 What success looks like 7

2 The Case for a Web Operating System 7
2.1 The browser has quietly become the computer 7
2.2 What a Web Operating System is, and what it is not 8
2.3 Why WebOS matters now 8
2.4 Why it is hard: the browser is a hostile host 9
2.5 A short history 9

3 Artificial Intelligence as an Operating-System Foundation 10
3.1 From application to primitive 10
3.2 What "AI-native" means in this project 10
3.3 The AI building blocks you will use 11
3.4 The AI trust boundary 11

Part II: The Brief

4 Project Logistics and Team Organisation 13
4.1 Team composition and engineering roles 13
4.2 Shared-ownership rules 13
4.3 Open technology stack, mandatory defence 14
4.4 Team contract 14

5 Name Your Operating System 14
5.1 Naming rules 14
5.2 The identity kit 15
5.3 Where your name must appear 15

6 The Boot Screen and Boot Sequence 16
6.1 Why your OS must boot 16
6.2 What the boot screen looks like 16
6.3 The boot stages 17
6.4 The service dependency graph 18
6.5 Boot requirements 18
6.6 Reference boot log 19

7 Industry Benchmarks and the Limitations to Overcome 20
7.1 Benchmark 1: Puter 20
7.2 Benchmark 2: daedalOS 21
7.3 Summary and verification 21

Part III: Technical Specification

8 Reference Architecture 22
8.1 Walk-through: one command, every layer 23

9 Operating-System Primitives 24
9.1 Process management and scheduling 24
9.2 Virtual memory and paging 26
9.3 The virtual file system 27
9.4 The window management subsystem 30
9.5 Offline edge execution and synchronisation 31
9.6 The system-call interface and the terminal 31

10 Data Structures and Algorithms Blueprint 32
10.1 Mandatory structures and where they live 32
10.2 Implementation policy 33
10.3 Interface sketches 33

11 Database Architecture and Integrity 34
11.1 Reference schema 34
11.2 Storing a tree in a database 35
11.3 ACID guarantees for bulk operations 35
11.4 Index optimisation 36
11.5 Database requirements 37

12 Foundational AI Integration 37
12.1 Choosing your subsystems 37
12.2 Subsystem A: semantic VFS search (RAG pipeline) 37
12.3 Subsystem B: agentic system execution 39
12.4 Subsystem C: predictive process pre-fetching 40
12.5 The coupling map 41
12.6 Models, cost and offline behaviour 41

Part IV: Delivery and Assessment

13 Milestones and Deliverables 42
13.1 Phase deliverables and gate criteria 42
13.2 Gate rules 43

14 The Architecture Defence 44
14.1 Structure of `STACK_DECISION.md` 44
14.2 Decisions that must be defended 44
14.3 What a strong justification looks like 45
14.4 The defence session 45

15 Repository, Documentation and Testing Standards 45
15.1 Repository layout 45
15.2 Required documents 46
15.3 Testing strategy 46
15.4 Commits and collaboration 47

16 Evaluation Rubric 47
16.1 Grading criteria and weights 47
16.2 Performance levels 48
16.3 Where the boot screen and identity are assessed 49
16.4 Individual moderation 49

17 The Final Defence 50
17.1 Demo Day protocol 50
17.2 Final deliverables 50

18 Academic Integrity and the Use of AI Tools 51

Appendices

A Consolidated Acceptance Checklist 51

B Glossary 52

C References and Further Reading 54

D Team and OS Registration Form 55

Part I

Why This Project Exists

Before writing any code, a team must understand what it is building and why. This part explains why web operating systems matter now and what separates a real operating system from an imitation of one. It also explains why artificial intelligence must be a foundation of the system rather than a feature placed on top of it.

Section 1

Executive Summary

Your team will design and build a complete Web-Based Operating System (WebOS) that runs natively inside a modern browser. It must boot through a genuine initialisation sequence and present a multitasking desktop. It must manage a hierarchical UNIX-like file system, schedule background processes on Web Workers, page idle application state out of memory, and persist everything with ACID guarantees. Most importantly, it must treat artificial intelligence as a subsystem of the operating system, not an application running on top of it.

The project is deliberately cross-disciplinary. Operating Systems, Data Structures & Algorithms, Database Systems and Artificial Intelligence stop being separate courses and become parts of one working system. Each concept you have studied has a mandatory place in this codebase:

- the process control block is a hash map;
- the scheduler is a heap;
- the directory hierarchy is a tree;
- the startup order is a topological sort of a dependency graph;
- a recursive delete is a database transaction;
- semantic search is cosine similarity between embeddings of your own files.

The project reproduces real systems-engineering constraints. The browser gives you one main thread and a storage quota that it is allowed to evict. It limits memory and will kill your tab if you exceed it, and the network can disappear at any moment. Your OS must stay fast within these limits, recover from failure and remain useful offline. Those are exactly the areas where the industry systems you will study, Puter and daedalOS, fall short.

> **The brief in one sentence**
>
> *Build an operating system, not a website that looks like one, in which intelligence is a system service.*

1.1 What your team will deliver

1. A named, bootable OS
Its own name, logo and version, a truthful boot sequence driven by a service dependency graph, a boot menu with safe and recovery modes, and a multi-user login screen.

2. A real desktop
A multi-window compositor with focus, z-ordering, dragging, resizing, minimising and a taskbar, holding 60 frames per second under load.

3. A UNIX-like file system
An inode tree with `/root`, `/home`, `/system` and `/bin`, full path resolution, read/write/execute permissions, soft links, hard quotas and distributed locking.

4. Processes and a scheduler
A process table and a priority scheduler that runs work on a pool of Web Workers, so the interface never freezes.

5. Virtual memory
An LRU pager that serialises idle application state and swaps it to local and then cloud storage, restoring it transparently when needed.

6. An ACID persistence layer
Atomic bulk operations that roll back cleanly when they fail midway, indexed metadata answering in under 10 ms, and a multi-tenant backend.

7. Foundational AI
At least two of: semantic search over the file system, an agent in the shell that runs verified OS commands, and predictive pre-fetching driven by a local model.

8. Engineering evidence
A written architecture defence, a test suite with fault injection and performance budgets, a production deployment, a technical report and a live final defence.

1.2 What success looks like

On Demo Day, an examiner who has never seen your system clones your repository and starts it. They boot it with the network disconnected, run a CPU-heavy job while dragging windows, and open more applications than memory can hold. They then pull the plug on a recursive delete halfway through and ask your OS, in plain language, to reorganise their files. A successful system keeps its frame rate, pages memory out and back in without losing a byte, and rolls back the failed delete with no orphaned directories. It also shows a plan and asks for confirmation before it touches anything. Each of those behaviours is specified precisely in Part III and listed as a checklist in Appendix A.

Section 2

The Case for a Web Operating System

2.1 The browser has quietly become the computer

For many people, most of the time, the operating system is now simply the thing that starts a browser. Email, documents, spreadsheets, design tools, code editors, video calls and a large share of enterprise software run as web applications. ChromeOS turned this into a product: a thin operating system whose main application is the browser.

Meanwhile, the browser has grown into a genuine runtime platform. It now offers:

- background threads (Web Workers);
- near-native computation (WebAssembly);
- GPU computation (WebGPU);
- a transactional database (IndexedDB) and a fast private file system (the Origin Private File System);
- offline execution (Service Workers);
- two-way networking (WebSockets and WebRTC).

A platform with threads, storage, a file system, networking and a security sandbox has the raw materials of an operating system. What it lacks is organisation: a process model, a scheduler, a memory manager, a file hierarchy with permissions, a window manager and a coherent notion of users. Providing that organisation is exactly what an operating system does, and it is what your team will build.

2.2 What a Web Operating System is, and what it is not

A Web Operating System is a software layer that implements classic operating-system abstractions on top of the browser runtime: processes, files, users, permissions, windows and services. Applications reach these abstractions through one uniform interface. A WebOS is *not* a website styled to look like a desktop. The difference is the same as the difference between an operating system and a screenshot of one:

Looks like an OS (not acceptable)	Behaves like an OS (required)
× Windows are `div`s whose position lives in scattered component state.	✓ Windows are entries in a compositor registry, and one authority manages their focus, z-order and lifecycle.
× Files are an array of objects saved to `localStorage`.	✓ Files are inodes in a tree with paths, permissions, links, quotas and transactional persistence.
× “Apps” are UI components rendered directly by the page.	✓ Applications are processes with IDs, priorities and memory budgets, and a lifecycle the kernel controls.
× Long tasks run on the main thread and freeze the interface.	✓ Long tasks run on Web Workers under a scheduler, and the interface never blocks.
× The boot screen is a CSS animation driven by a timer.	✓ The boot screen shows real services starting in dependency order.
× AI is a chat panel in a corner of the screen.	✓ AI is a system service that indexes the file system, issues verified system calls and advises the scheduler.

2.3 Why WebOS matters now

Zero-install, device-independent computing
The whole environment is a URL. Any device with a modern browser becomes a workstation, and a user's files, windows and settings follow them from machine to machine.

Low-cost and shared hardware
University labs, schools, offices and homes often share modest machines. A WebOS gives every user a centrally managed, always-updated personal desktop on inexpensive hardware, with nothing installed locally.

Security by construction
The browser isolates origins in a sandbox, and updates are atomic deployments. Users need no administrator rights. A WebOS inherits these guarantees and adds its own permission model on top.

Collaboration as a default
Multi-user state, real-time synchronisation and shared file systems come naturally on the web but are awkward to build into a single-machine OS.

Offline-first edge computing
Service Workers and local storage let a web system keep working without a connection. That discipline matters wherever connectivity is intermittent or expensive.

A natural home for AI
Models, data and users already meet on the web. An OS that can see a user's files (with permission) and act on them is the right place to host an assistant that does real work.

2.4 Why it is hard: the browser is a hostile host

A browser was designed to display documents safely, not to host an operating system. Every classic OS responsibility runs into a browser limitation. This project exists to make you engineer around those limitations rather than ignore them.

Browser constraint	Consequence if ignored	What your OS must do
One main thread renders the interface *and* runs your JavaScript.	Any task over about 50 ms drops frames, and heavy work freezes the whole desktop.	Run work on Web Workers under a scheduler and keep the kernel's main-thread share small (§9.1).
Tab memory is finite. The browser may discard or crash a tab under memory pressure.	Running several heavy applications at once crashes the session. This is the daedalOS problem.	Active memory paging: LRU eviction of inactive application state to swap (§9.2).
Storage is quota-managed, and best-effort storage may be evicted by the browser.	Local data is not durable by default.	Request persistent storage, monitor the quota, and page out to the backend (§9.2, §11).
There is no native file hierarchy, ownership or permission model.	Without real files, the “OS” is only a demo.	A virtual file system with inodes, paths, permissions and quotas (§9.3).
The network is unreliable.	A cloud-dependent system stops working. This is the Puter problem.	Offline edge execution with a replayable operation log (§9.5).
A running JavaScript function cannot be interrupted from outside.	Classic pre-emptive scheduling cannot be copied literally.	Time-sliced cooperative yielding, plus forced termination of runaway processes (§9.1).
Several tabs, devices and users can change the same data at the same time.	Lost updates and corrupted directory trees.	Transactions, leases, locks and version numbers (§9.3, §11).

2.5 A short history

The idea of a network-delivered computer is older than the web. Each generation below solved one part of the problem this project asks you to solve as a whole.

Era	System or idea	What it contributed
1960s–1970s	Time-sharing systems and terminals	Many users sharing one central computer through thin terminals; the original “cloud”.
1990s	Network computers and thin clients	Diskless machines that load their software and data from a server.
2000s	Early web desktops (e.g. eyeOS) and rich web applications	Showed that a desktop, with windows and files, can be drawn inside a browser.

Era	System or idea	What it contributed
2009–2011	ChromeOS and the first Chromebooks	Commercial proof that the browser can be the main computing environment.
2017	WebAssembly in all major browsers	Near-native speed in the page, so compilers, emulators and databases can run client-side.
Late 2010s–2020s	OPFS, WebGPU, in-browser runtimes; web desktops such as Puter and daedalOS	Real file-system APIs, GPU computation and complete open-source web desktops.
Today	On-device and cloud language models	Intelligence becomes an operating-system service. That is the step this project asks you to take.

Section 3

Artificial Intelligence as an Operating-System Foundation

3.1 From application to primitive

Operating systems have a long history of absorbing capabilities that started as applications. Networking was once an add-on, and TCP/IP now ships in every kernel. Full-text search was once a separate program, and file indexing is now an OS service that any application can query. Each time, the capability moved into the OS for two reasons. It needed privileged, system-wide access to every file, process and event. And every application benefited from sharing one trustworthy implementation.

Artificial intelligence is making the same move. Major desktop and mobile platforms now ship language models as system services rather than stand-alone apps, and browsers have begun to offer on-device models to web pages. The reason is structural: a model is only as useful as the context it can see and the actions it can take, and the operating system is where both live.

That is why this specification forbids a cosmetic chatbot. A chat panel that sees only the text you paste into it, and replies only with text, is an application. An AI subsystem that is part of the operating system does five things:

- it is notified when files change;
- it indexes them as a background process;
- it answers questions by searching the file system by meaning;
- it turns intent into verified system calls;
- it tells the scheduler what the user is about to need.

3.2 What “AI-native” means in this project

In this project, an AI-native operating system has four properties. Your design **must** show all four for every AI subsystem you build.

1. AI sees the system through the kernel
AI services read files through the same system calls as any other process and pass the same permission checks. They never read storage directly. System events (file writes, process exits, application launches) reach them on the kernel's event bus.

2. AI acts through system calls, not strings
An agent produces a structured plan of typed operations, and the kernel validates the plan before anything runs. Model output is parsed as data. It is never passed to `eval` or concatenated into a shell string.

3. AI is scheduled by the system
Embedding, indexing, training and inference run as processes with IDs, priorities and memory budgets. They appear in `ps` and can be killed, and the scheduler throttles them while the user is active.

4. AI informs the system
Predictions feed back into OS decisions: which applications to pre-load, which memory pages to keep resident, and which background jobs to run first.

3.3 The AI building blocks you will use

Building block	What it is	Where it lives in your OS
Embeddings	A model maps a piece of text to a vector of numbers so that texts with similar meaning produce nearby vectors.	The VFS indexer embeds each file chunk when the file is written.
Vector similarity	Cosine similarity measures the angle between two vectors: cos θ = (a·b) / (‖a‖‖b‖).	Semantic search ranks file chunks against a query (§12.2).
Retrieval-augmented generation (RAG)	Relevant documents are retrieved first and given to a language model, so its answer is grounded in real sources and can cite them.	Answers about the user's own files, with citations to paths.
Tool (function) calling	A language model emits structured calls to functions described by JSON schemas instead of free text.	Each agent tool is one OS system call with a typed schema (§12.3).
Agents	A loop in which a model plans, a runtime validates and executes, and the result is observed before the next step.	The shell's natural-language command mode, with confirmation and rollback.
Small predictive models	Markov chains, naive Bayes, decision trees or small neural networks trained on usage telemetry.	Predicting the next application launch and pre-fetching it (§12.4).
On-device inference	Running models inside the browser through WebAssembly or WebGPU runtimes, with no server round trip.	Embedding and prediction inside Web Workers, which also keeps AI working offline.

3.4 The AI trust boundary

An AI that can act on a file system can also destroy one. The rigour that protects the kernel from applications must also protect the system from the model. **The model is an untrusted user-space process: it proposes, and the kernel decides.** Every AI subsystem you build **must** respect these principles:

- **Least privilege.** The agent runs with the invoking user's identity, never as root, and cannot raise its own privileges.
- **Plan before acting.** Every multi-step action first produces a plan the user can inspect, and a dry run that lists exactly which files it will affect.

- **Confirmation.** Destructive or irreversible operations need explicit user confirmation.
- **Atomicity.** A confirmed plan runs as one transaction or not at all. A failure rolls everything back.
- **Content is data, never instruction.** File contents and file names in the model's context may contain prompt-injection attacks. They must never be able to trigger a tool call on their own authority.
- **Auditability.** Every AI-initiated system call is logged with its prompt, plan, user and outcome.
- **Graceful absence.** When the model is unavailable (offline, rate-limited or failing), the OS keeps working and AI features degrade with a clear message.

The litmus test

Remove your AI module and rebuild. If only a sidebar disappears, your AI is cosmetic and will be graded as such.

If the scheduler loses its pre-fetch hints, the shell loses natural-language execution and the file system loses semantic search, while everything else keeps working, then your AI is foundational and correctly isolated. That is the target.

Part II

The Brief

Who builds the system and what it is called. How it comes to life when a user opens it, and which existing systems it must outperform. This part defines the team, the operating system's identity, the mandatory boot screen and boot sequence, and the industry benchmarks your design must answer.

Section 4

Project Logistics and Team Organisation

4.1 Team composition and engineering roles

Each team has **exactly five students**. Teams of any other size are not accepted without written approval from the instructor. Each member holds one of the five engineering roles below as their *primary* ownership. The roles are designed so that every member owns production code in a real subsystem; no role is limited to documentation or testing.

Role	Primary ownership	DSA / DB responsibility	Key evidence produced
1. Kernel / Systems Architect	Process table and control blocks, the scheduler, the Web Worker pool, the system-call and IPC bridge, the memory manager and LRU pager.	Binary heap; PCB and page-table hash maps; LRU list.	ARCHITECTURE.md; scheduler and paging benchmarks.
2. VFS / Database Engineer	The VFS tree, path resolution, permissions, links and quotas; the database schema, transactions and indexing; the sync and distributed-lock backend.	N-ary tree; B-tree and hash indexes; file-handle table.	DATABASE.md; fault-injection and query benchmarks.
3. Window Manager / UI Subsystem Lead	Boot, login and desktop screens; the compositor and taskbar; the terminal interface with autocomplete; built-in application interfaces; accessibility and frame rate.	Window registry; z-order structure; trie.	Frame-time evidence; UI and accessibility audit.
4. AI Pipeline Specialist	The embedding pipeline and vector search, the agentic shell (planner, validator, executor), the pre-fetch model, evaluation sets and AI safety.	Vector index; telemetry model (e.g. a Markov chain).	AI.md; evaluation reports; safety test results.
5. QA / DevOps & Integration Engineer	The service manager and boot orchestration, offline execution (Service Worker and operation log), CI/CD, the test and performance harness, fault injection, deployment and observability.	DAG and topological sort; ring buffer for logs.	BOOT.md; TESTING.md; CI pipeline; deployment.

4.2 Shared-ownership rules

- Every subsystem has a **primary owner** and a named **secondary reviewer**, so no subsystem depends on a single person.

- Every pull request is reviewed and approved by at least one other member before it is merged. Direct pushes to the main branch are disabled.
- Every member **must** author meaningful code in **at least two subsystems**: their own and one other.
- Every member is examined on the **whole system** at each gate and at the final viva, not only on the part they wrote.

4.3 Open technology stack, mandatory defence

You may use any language, framework, runtime or database. That includes React, Next.js, Svelte, Vue, plain Web Components, TypeScript, Rust or Go compiled to WebAssembly, Node.js, Deno, Bun, Go, Python, PostgreSQL, SQLite, MongoDB, Neo4j, Redis and any vector store.

That freedom comes with a price. **Arbitrary technology choices are penalised.** Each major choice must be defended in writing against the performance and integrity constraints of a WebOS, following the procedure in Section 14. “We already knew it” is a legitimate factor in a decision matrix, but it is never a justification on its own.

4.4 Team contract

Each team signs a team contract in Week 2 and commits it to the repository as `TEAM_CONTRACT.md`. It **must** state:

- the role and secondary-review assignments;
- the team's definition of “done” for a task;
- the meeting and communication schedule, and how quickly each member commits to reply;
- how the team makes decisions when members disagree;
- what the team does, and by when, if a member stops contributing, so the problem is handled early rather than in Week 16;
- a signed acknowledgement that marks are moderated individually using the viva, commit history and peer assessment (Section 16.4).

Section 5

Name Your Operating System

Every operating system has an identity: a name, a version, a logo, a boot screen and an About dialog. Naming your system is not decoration. It makes the team agree on what the product is, and it gives every artefact (the repository, deployment, report and demo) one consistent identity.

Each team **must** choose a decent, original name for its WebOS and register it using the form in Appendix D by the end of **Week 2**. The name then appears everywhere the system identifies itself.

5.1 Naming rules

Original	The name must not belong to an existing operating system or major software product (for example Windows, macOS, ChromeOS, Linux, Ubuntu, Android, Puter or daedalOS). It must not be confusingly similar to a registered trademark. The generic term “WebOS” used in this document is not a valid name, because webOS is also an existing commercial product.

Decent and professional	The name must be suitable to print on a university report and present to industry judges. It must not contain profanity, slurs, innuendo, political, sectarian or religious references, or the names of real people.
Short and pronounceable	3 to 16 characters in the Latin alphabet. It may end in "OS" (for example *<Name>OS* or *<Name> OS*). Test it by saying it aloud: *"Boot into ___ and open the terminal."*
Unique in the cohort	Names are registered first come, first served, and a name already taken by another team is rejected. This document deliberately suggests no names: choosing one is your team's task.
Stable	After Phase 1 the name can be changed only with the instructor's written approval.

Where good names come from

Strong names often come from nature and astronomy (a star, a mountain, a bird), from local heritage and geography (a river, a landmark, a historic city), or from a coined word that hints at what makes your system different. The best names are short, easy to spell after hearing them once, and fit your logo.

5.2 The identity kit

By the Phase 2 gate, your repository **must** contain an identity kit in `/system/branding` and in `docs/`:

Item	Requirement
Name and wordmark	The registered name, written the same way everywhere, with capitalisation and spacing fixed.
Logo	An SVG logo that is legible at 16 px (favicon and taskbar) and at 256 px (boot screen).
Version scheme	Semantic versioning (`MAJOR.MINOR.PATCH`), starting at `0.1.0`. Each phase ends with a tagged release: v0.2.0 at Phase 2, v0.3.0 at Phase 3, v0.4.0 at Phase 4 and v1.0.0 at the final defence. Release codenames are optional.
Tagline	One line that states your system's distinctive idea, for example what it does that Puter and daedalOS do not.
Design tokens	Colours, typography, spacing and motion values in one tokens file, used by every system screen. Light and dark themes.
Kernel identifier	The string printed by `uname`, including the name, version, short build hash and build date.

5.3 Where your name must appear

Boot and login

Boot screen, boot menu, recovery console and login screen.

System surfaces

Settings → About dialog (name, version, build hash, build date and team members), browser tab title and favicon, installable app manifest.

Artefacts

Repository name, README header, technical report cover, slides and demo video title.

```
student@your-os:~$ uname -a
YourOS 0.3.0 (kernel 0.3.0-a1f93c2, built 2026-11-02) · Chrome 153 · 8 cores ·
uid=1001(student)
student@your-os:~$ cat /system/release
NAME="YourOS"   VERSION="0.3.0"   BUILD="a1f93c2"
TEAM="Section D · Team 04"
```

Section 6

The Boot Screen and Boot Sequence

6.1 Why your OS must boot

When a physical computer is switched on, several stages run in order:

1. firmware performs a power-on self-test (POST);
2. a bootloader selects and loads a kernel;
3. the kernel initialises its core data structures;
4. an init system such as *systemd* starts services in dependency order until the login screen appears.

Each stage exists because something must be ready before something else can run.

Your WebOS needs a boot sequence for the same reason. Before the desktop can appear, several things must happen in order:

- the kernel creates the process table and starts the scheduler;
- storage is opened, and the file system is mounted and checked;
- the user is authenticated;
- background services (synchronisation, locking, AI indexing and prediction) start in an order that respects their dependencies.

The boot screen is the visible face of this process. **It is mandatory, and it must tell the truth**: every movement of the progress indicator must correspond to real work finishing.

> **Not acceptable**
> A progress bar driven by `setTimeout` or `setInterval`, a fixed-length animation, or a splash screen that hides a desktop that has already loaded. A boot screen that pretends to load is graded as absent.

6.2 What the boot screen looks like

The visual design of your boot screen is yours to choose: minimal, retro BIOS, cinematic, or something new. What is fixed is the *information* it shows and the *honesty* of its progress.

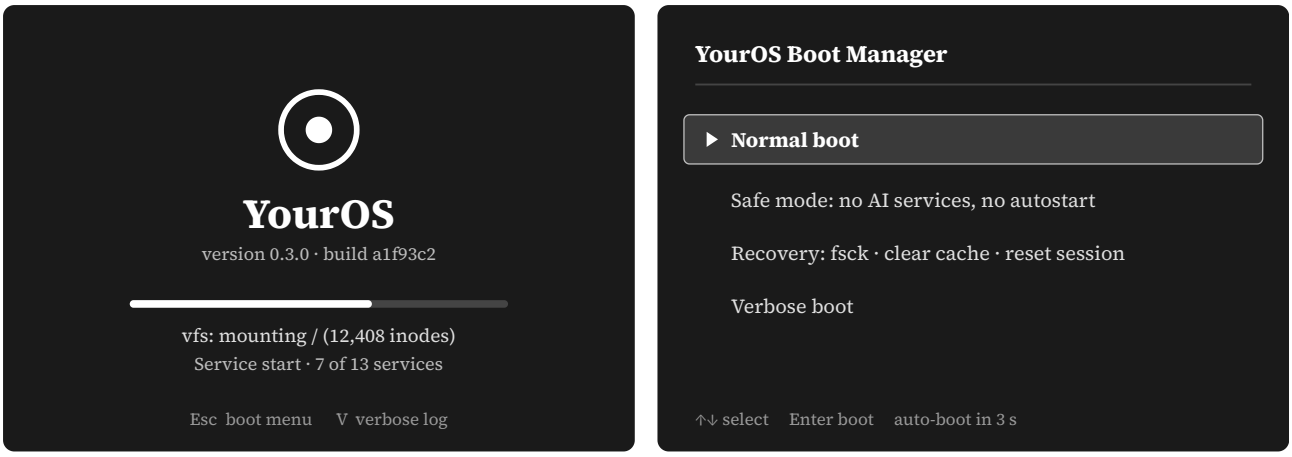


Figure 6.1 — Illustrative boot splash (left) and boot menu (right). The splash must show the name, version, truthful progress, the current service and how to reach the boot menu and verbose log. “YourOS” stands for your team's registered name.

6.3 The boot stages

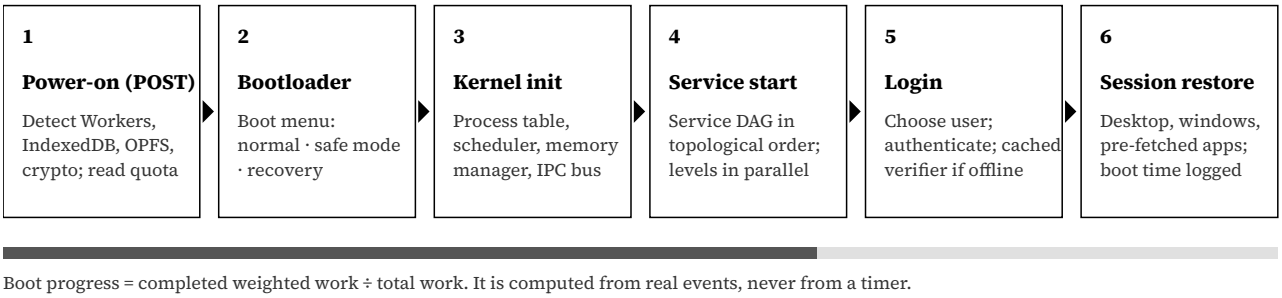


Figure 6.2 — The six boot stages. Stages 3 and 4 are driven by the service dependency graph in Figure 6.3. Stage 5 must still work when the device is offline (BOOT-09).

Stage	What happens	Shown on screen	If it fails
1. POST	Detect required browser capabilities; check the Service Worker cache; read the storage estimate and request persistent storage.	Logo; “checking system”.	“Unsupported browser” screen listing the missing capability.
2. Bootloader	Read boot flags; show the boot menu if requested (e.g. Esc held); choose the profile.	Boot menu (Figure 6.1).	Fall back to safe mode.
3. Kernel init	Create the process table (PID 1 = `init`), start the scheduler and worker pool, arm the memory manager, open the IPC bus.	Progress; verbose log lines.	Kernel panic screen with diagnostic details.
4. Service start	Load service manifests, build the DAG, reject cycles, and start services level by level.	Current service and count.	Critical service: recovery screen. Non-critical: continue degraded.
5. Login	Select a user and authenticate against the server, or against a cached verifier when offline.	Login screen with users.	Offer guest or offline mode.
6. Session restore	Restore windows and layout, start pre-fetched applications, record boot timings.	Desktop.	Start a clean session and keep the old one for recovery.

6.4 The service dependency graph

Each system service declares its dependencies in a manifest. At boot the service manager:

1. builds a directed graph from the manifests;
2. **rejects cycles**;
3. computes a topological order with Kahn's algorithm;
4. starts every service in the same level concurrently.

Shutdown runs the same order in reverse. The boot progress bar is the weighted fraction of graph nodes that have finished starting.

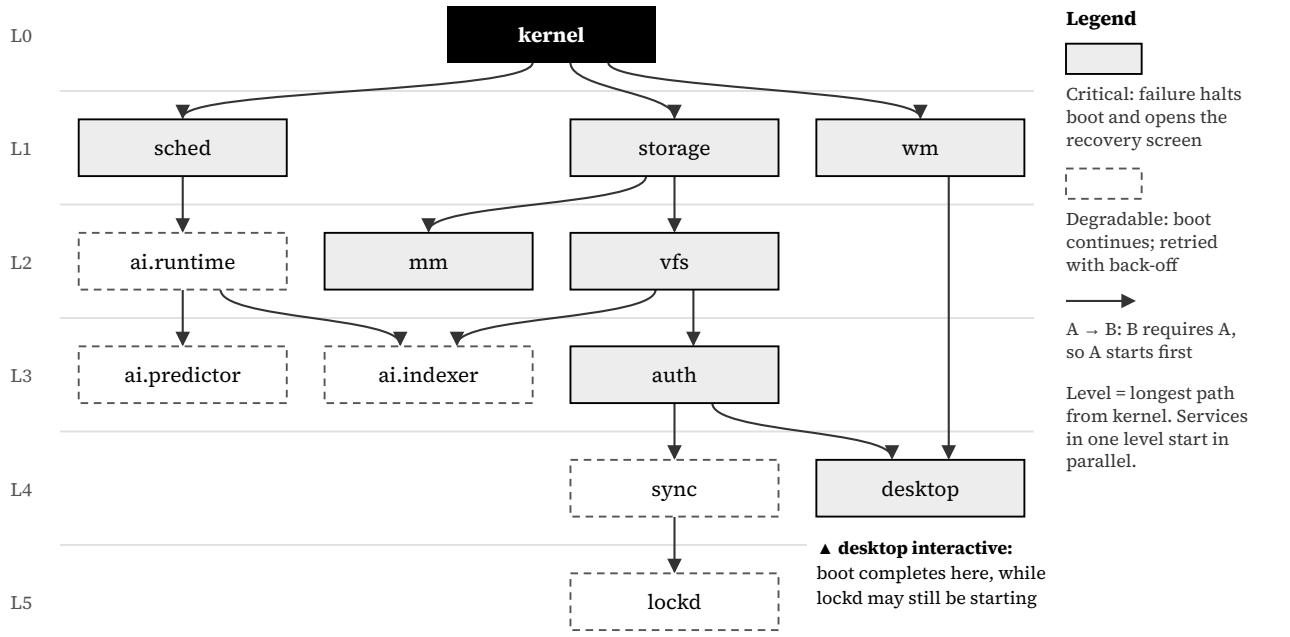


Figure 6.3 — An example service DAG with 13 services in six levels. Your services and edges will differ. What is fixed is the mechanism: the order comes from topological sorting, levels run in parallel, cycles are rejected, and only critical services can block the desktop.

```
// /system/services/ai.indexer.json (comments added for illustration)
{
  "name": "ai.indexer",
  "requires": ["ai.runtime", "vfs"],
  "critical": false,     // failure degrades instead of halting the boot
  "weight": 3,           // share of the boot progress bar
  "timeoutMs": 4000,
  "priority": 32,        // scheduler priority once running (0 = highest, 39 = lowest)
  "safeMode": false      // not started in safe mode
}
```

6.5 Boot requirements

ID	Requirement	How it is verified
BOOT-01	**Must:** The boot screen displays the registered OS name, logo and version.	Visual inspection at the Phase 2 gate.

ID	Requirement	How it is verified
BOOT-02	**Must:** Progress is computed from real work finishing (weighted DAG nodes and asset loading). No timer-driven progress anywhere in the boot path.	Code review. On a throttled network, the progress bar must slow down in step with the network.
BOOT-03	**Must:** Services start in the topological order of the service DAG, and independent services start concurrently.	`bootlog` timestamps show levels overlapping.
BOOT-04	**Must:** A dependency cycle halts the boot with a kernel-panic screen that names the cycle.	Examiner adds a cycle to a manifest.
BOOT-05	**Must:** A verbose mode (toggled by a key) shows a timestamped, kernel-style boot log.	Live demonstration.
BOOT-06	**Must:** A boot menu offers *Normal*, *Safe mode* (no AI services, no autostart apps, default theme) and *Recovery* (file-system check, clear local cache, reset session).	Live demonstration.
BOOT-07	**Must:** If a non-critical service fails or times out, the boot continues; the desktop shows which services are degraded; failed services are retried with back-off.	AI endpoint or backend blocked during boot.
BOOT-08	**Must:** If a critical service fails, a recovery screen offers retry, safe mode and diagnostic details, never a blank page or an endless spinner.	Local database version corrupted by the examiner.
BOOT-09	**Must:** With the network disabled, the system boots to a usable desktop from cached assets and local state.	Browser set to offline, then reload.
BOOT-10	**Must:** Warm boot (cached) ≤ 1.5 s and cold boot ≤ 3.0 s to an interactive desktop. Large assets such as model weights load only after the desktop is interactive.	Performance trace on the reference machine.
BOOT-11	**Must:** Per-stage and per-service timings are saved. `bootlog` prints them, and `bootlog --critical-path` shows the chain of services that determined the boot time.	Terminal demonstration.
BOOT-12	**Must:** Graceful shutdown and restart: dirty pages and pending writes are flushed, services stop in reverse topological order, and the session state is saved. Restarting returns to the same session.	`shutdown -r now` during a live session.
BOOT-13	**Must:** The boot screen is accessible: it respects `prefers-reduced-motion`, exposes `role="progressbar"` with live values, announces status changes to screen readers, and the boot menu works from the keyboard alone.	Accessibility audit.
BOOT-14	**Should:** The login screen supports several users and includes a guest or demo account for examiners.	Live demonstration.

6.6 Reference boot log

Your verbose boot log does not have to match this format, but it must contain the same information. The example below shows a boot on a flaky network: the sync service is degraded and the desktop still comes up.

```
[    0.000] firmware: YourOS 0.3.0 · POST ok (workers, indexeddb, opfs, crypto) · quota 2.1 GB,
persisted
[    0.004] loader: profile=normal · service worker active (offline-capable)
[    0.011] kernel: process table ready (hash map, cap 1024) · pid 1 = init
[    0.013] sched: priority heap + RR · quantum 8 ms · workers 7/8 cores · aging 250 ms
[    0.019] mm: memory manager armed · budget 256 MB · swap tiers: idb, cloud
[    0.021] svc: 13 manifests · 14 edges · acyclic · 6 levels
[    0.087] storage: opened youros-vfs v7 (IndexedDB) in 64 ms
[    0.212] vfs: mounted / · 12,408 inodes · 3 users · quotas enforced
[    0.215] vfs: fsck clean · 0 orphans · 0 dangling links
[    0.341] auth: session restored for uid=1001 (student)
[    0.402] sync: WARN backend unreachable · offline mode · outbox holds 3 ops · retry in 2 s
[    0.405] lockd: DEGRADED (requires sync) · local advisory locks only
[    0.644] ai.indexer: 412 chunks pending · scheduled at priority 32 (background)
[    0.651] ai.predictor: model loaded (Markov, 214 transitions) · pre-fetch: Terminal, Files
[    0.731] desktop: interactive · 4 windows restored · boot 0.73 s · 2 services degraded
```

And the only acceptable response to a dependency cycle:

```
[    0.021] svc: 13 manifests · 15 edges
[    0.021] svc: KERNEL PANIC: dependency cycle detected: sync → lockd → auth → sync
            boot halted. [R] recovery console   [S] safe mode   [D] dump diagnostics
```

Section 7

Industry Benchmarks and the Limitations to Overcome

You are not building in a vacuum. Two open-source web operating systems set the bar. Your design must study both, measure where they fall short, and show how your architecture overcomes each limitation.

7.1 Benchmark 1: Puter

Architecture. Puter is a full-featured, browser-based desktop environment. It combines a cloud-backed storage layer, hosting for web applications, and a developer API through which applications use storage and hosted services, including AI.

Limitation: centralised cloud coupling	**Your required response**
Core functions depend on a continuous connection to the remote server. When the network drops, the environment loses most of its usefulness.	**Offline edge execution.** The core utilities (terminal, file explorer, text editor, task manager and settings) must work fully offline against a local-first VFS. Changes queue in an operation log and are replayed when the connection returns (§9.5).
Limitation: superficial AI layer	**Your required response**
AI is offered as a service that applications call, like a widget. It is not part of how the system schedules work, stores files or runs commands.	**AI woven into OS primitives.** AI jobs are scheduled processes, retrieval is a file-system capability, and natural language compiles into validated system calls in the shell (§12).

7.2 Benchmark 2: daedalOS

Architecture. daedalOS is a client-centric web desktop that recreates the look and feel of a local desktop entirely client-side. Its file system is built on BrowserFS, which layers a Node-style file-system API over browser storage. It runs applications and emulators directly in the page.

Limitation: memory bloat
Running heavy applications or emulators at the same time keeps all their state in one tab, which can exhaust memory and crash the browser.

Your required response
Active memory paging. A memory budget, LRU eviction of inactive application state, and two-tier swap to local storage and backend cloud storage, with transparent page-in (§9.2).

Limitation: lack of orchestration
There is no robust multi-tenant backend, so there is no distributed file locking and no synchronisation between users or devices.

Your required response
A multi-tenant backend. Per-user storage and quotas, lease-based distributed file locks, versioned synchronisation and conflict handling over WebSockets (§9.3, §9.5, §11).

7.3 Summary and verification

Benchmark	Limitation	Your required response	How it is verified at the final defence
Puter	Cloud coupling	Offline edge execution for core utilities; local-first VFS with replayable log.	Network disabled; boot; create, edit, move and delete files; reconnect; the server state converges with no lost operations.
Puter	Superficial AI	AI inside the scheduler, file system and shell.	AI processes shown in `ps`; the litmus test of §3.4; agent plans executed through system calls.
daedalOS	Memory bloat	LRU paging with local and cloud swap.	Open a workload of twice the memory budget; memory stays within budget; restored state matches its checksum.
daedalOS	No orchestration	Multi-tenant backend with locking and sync.	Two users on two machines contend for one file; the lock is honoured; the lease expires after a crash.

Phase 1 task: benchmark analysis
Run both systems yourselves. Measure memory use with several applications open (the browser's task manager is enough), observe what happens offline, and record what you find. Submit a two-page benchmark analysis with the Phase 1 dossier, with screenshots and numbers as evidence. Claims copied from websites without measurement earn no credit.

Part III

Technical Specification

This is the engineering core of the document. It starts with a reference architecture, then specifies each subsystem in turn: the operating-system primitives, the mandatory data structures, the database and its integrity guarantees, and the foundational AI layer. Every testable requirement has an identifier.

Section 8

Reference Architecture

The architecture below is a *reference*, not a prescription. You may restructure it, merge or split components, or choose different technologies. However, your architecture **must** keep three boundaries:

1. **Applications reach OS resources only through the system-call interface.** No application, including built-in ones, touches storage, the process table or another application's windows directly.
2. **The main thread only renders and dispatches.** Computation runs on workers. The kernel's own work on the main thread is small and bounded.
3. **The client can work alone, and the server is the authority on shared state.** The client runs offline from local state. The backend is authoritative for multi-user data, locks and quotas.

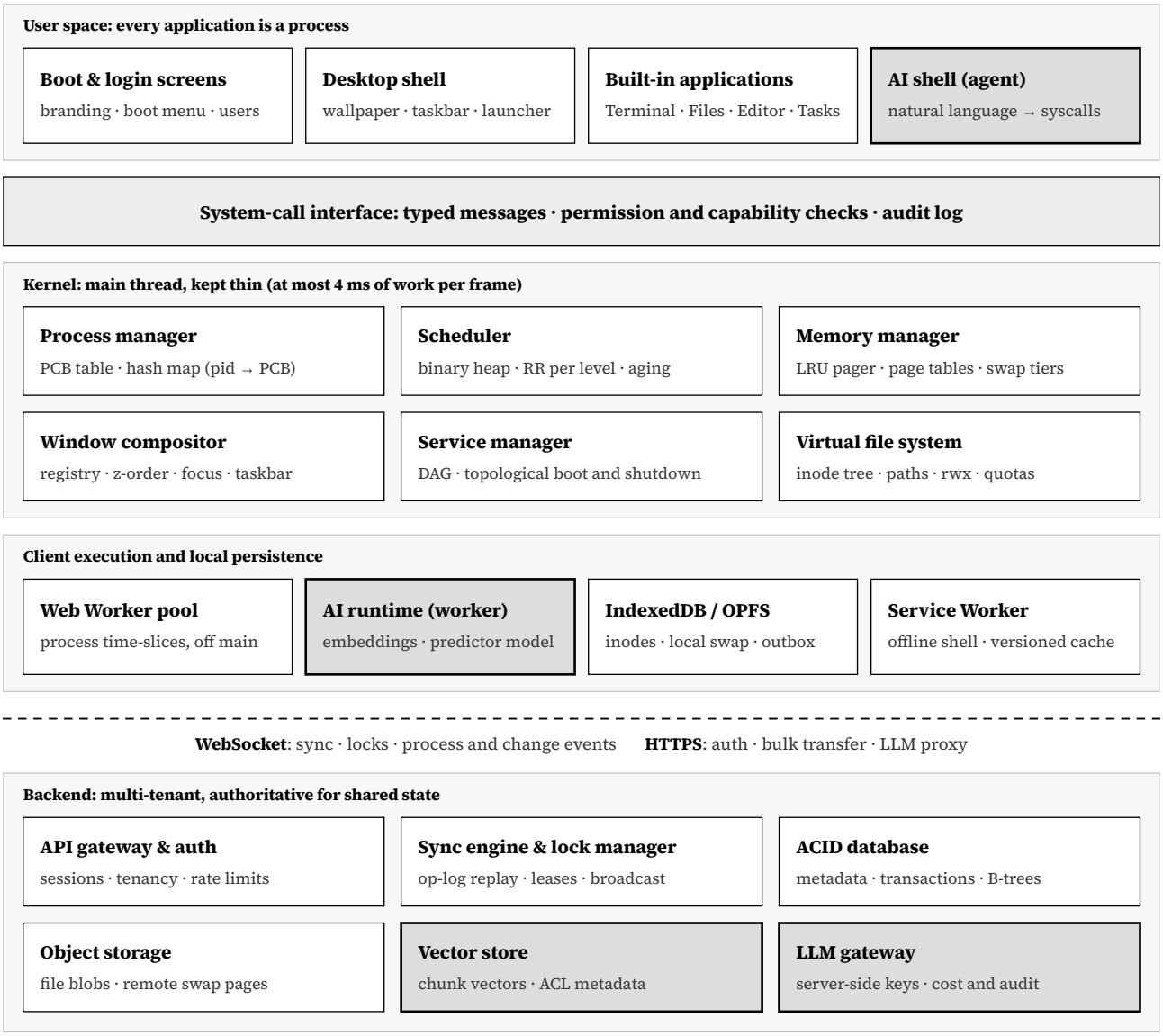


Figure 8.1 — Layered reference architecture. Shaded components belong to the AI subsystem. They appear in *every* layer, from the shell to the kernel's workers to the backend. That is what "woven into the OS" looks like in a diagram.

8.1 Walk-through: one command, every layer

To check that your architecture holds together, trace one command through it. Here is `rm -r ~/old-drafts` typed into the terminal:

1. The terminal process parses the command and issues `sys.unlink(path, {recursive: true})` through the system-call interface.
2. The kernel resolves the path in the VFS tree and checks write permission on the parent directory and every descendant.
3. The VFS applies the subtree deletion to the local store in **one** IndexedDB transaction, and appends one operation, with an idempotency key, to the outbox.
4. The sync client sends the operation over the WebSocket. The server acquires locks, runs the recursive delete in one database transaction, updates the quota and commits. Any failure rolls back.
5. The server broadcasts the change to the user's other sessions. The AI indexer, a background process, is notified and removes the deleted files' vectors.

6. The terminal prints the result. If the server rejected the operation, the client reverts its local change and shows the reason.

Section 9

Operating-System Primitives

This section specifies the six primitives that make your system an operating system: processes and scheduling, virtual memory, the file system, window management, offline execution, and the system-call interface with its terminal.

9.1 Process management and scheduling

The process model

Everything that runs is a process: applications, system services, AI jobs and terminal commands. The kernel keeps a **process table**, a hash map from process ID to process control block (PCB). Every lookup, insert and removal is O(1) on average.

PCB field	Meaning
`pid`, `ppid`	Process ID and parent process ID. PID 1 is `init`, which adopts orphaned processes.
`uid`, `gid`	Owner identity, used for every permission check the process triggers.
`state`	One of NEW, READY, RUNNING, BLOCKED, SUSPENDED (paged out) or TERMINATED (Figure 9.1).
`priority`, `nice`	Static priority from 0 (highest) to 39 (lowest), adjusted by `nice`; the effective priority also includes aging.
`cpuTimeMs`, `lastRunAt`, `waitingSince`	Accounting for `top`, time slices and aging.
`worker`	Handle of the Web Worker currently executing the process, if any.
`pageTable`, `residentBytes`	Memory-manager data (§9.2).
`fdTable`	Open file descriptors: a hash map from file descriptor to open-file handle.
`waitingOn`, `exitCode`	The event or resource a blocked process is waiting for, and the exit status.

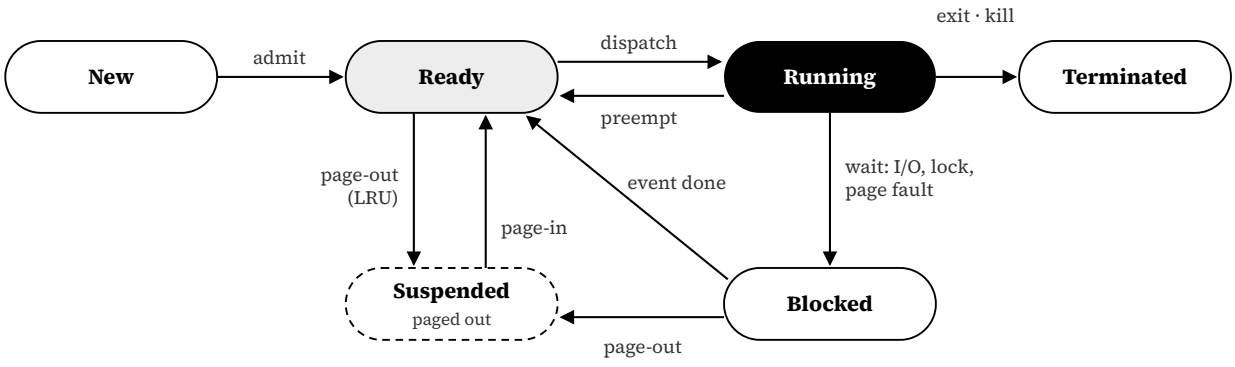


Figure 9.1 — Process states. SUSPENDED is the state that links the scheduler to the memory manager: a process whose state has been paged out cannot be dispatched until a page-in brings it back to READY.

The scheduler

- **Ready queue.** A binary min-heap that *you implement*. Its key is (effective priority, arrival sequence number). The sequence number gives FIFO order within a priority level, which produces **round-robin within each priority**.
- **Aging.** A process that waits for longer than a set interval has its effective priority improved step by step. No runnable process starves, and you must state and prove the bound.
- **Time slices.** Long-running work is written as resumable steps (generators, chunked loops or explicit state machines). A worker runs one *quantum* of steps, reports back, and the scheduler re-queues the process.
- **Pre-emptive and non-pre-emptive classes.** Interactive and foreground processes pre-empt background ones at the next slice boundary. Non-pre-emptive tasks, such as a transaction commit, run to completion. A process that exceeds its hard limit is forcibly terminated with `worker.terminate()`.
- **Multi-core dispatch.** The worker pool is sized from `navigator.hardwareConcurrency`, and the scheduler maps runnable processes onto free workers.
- **Main-thread budget.** The kernel's bookkeeping must stay small enough that the interface holds 60 FPS, which is a frame budget of 16.7 ms shared with rendering.

> **Engineering reality: pre-emption in a browser**
> A running JavaScript function cannot be interrupted from outside. You therefore implement pre-emption at two levels. *Cooperative time-slicing* at yield points handles the normal case. *Forced termination* handles runaway processes. Explaining this trade-off clearly, and what it costs a process that is terminated mid-slice, is part of your defence.

ID	Requirement	How it is verified
PROC-01	**Must:** The process table is a hash map keyed by PID, with O(1) average lookup, insert and delete.	Code review; unit tests.
PROC-02	**Must:** The ready queue is your own binary heap with O(log n) push and pop, and O(log n) priority change through an index map.	Unit tests; micro-benchmark.
PROC-03	**Must:** Round-robin among equal priorities with a configurable quantum.	Scheduling trace test.
PROC-04	**Must:** Aging guarantees that no runnable process waits longer than a declared bound, whatever its priority.	Starvation test: 50 high-priority CPU jobs and 1 low-priority job.
PROC-05	**Must:** All long-running work runs on Web Workers. With a CPU-bound workload running and 10 windows open, the p95 frame time is ≤ 16.7 ms, and no long task over 50 ms comes from OS code.	Performance trace; `PerformanceObserver` long-task log.
PROC-06	**Must:** The process lifecycle follows Figure 9.1, and every state transition is logged.	Trace inspection.
PROC-07	**Must:** `kill` terminates the process, frees its worker, closes its file descriptors and releases its locks. Its children are re-parented to `init`.	Integration test.
PROC-08	**Must:** `ps`, `top`, `kill`, `nice`, `renice`, `jobs`, `bg`, `fg` and a trailing `&` work, and a graphical Task Manager shows state, CPU share and memory.	Live demonstration.

ID	Requirement	How it is verified
PROC-09	**Should:** The process table holds at least 256 concurrent processes (most of them sleeping) without degrading the interface.	Load test.

9.2 Virtual memory and paging

Your OS is responsible for the memory of every application it runs. Each process has in-memory state: documents, canvases, editor buffers, emulator state and caches. When the total resident state exceeds the system's **memory budget**, the memory manager must reclaim memory in three steps:

1. serialise the least recently used inactive state;
2. write it to swap;
3. release it from memory.

When the process is needed again, the state must be paged back in without the user having to do anything.

Memory accounting

Browsers do not report exact memory use per object, so your team **must** define and document an *accounting model*. A typical model counts:

- the serialised size of each process's state;
- the declared size of its large assets (blobs, image bitmaps, WebAssembly memory).

Check your model against real measurements, for example the browser task manager or `performance.measureUserAgentSpecificMemory()` where it is available (it requires cross-origin isolation), and report the error.

Pages, the page table and LRU replacement

Each process's state is divided into **pages**, either fixed-size chunks of serialised state or logical units such as one document. A per-process **page table** maps each page to its location (RESIDENT, LOCAL_SWAP or REMOTE_SWAP), with a dirty bit, a last-access time, a size and a checksum. Replacement uses a global **LRU list**, built from a hash map and a doubly linked list so that touching a page and evicting a page are both O(1). The focused process and pinned system processes are never evicted.

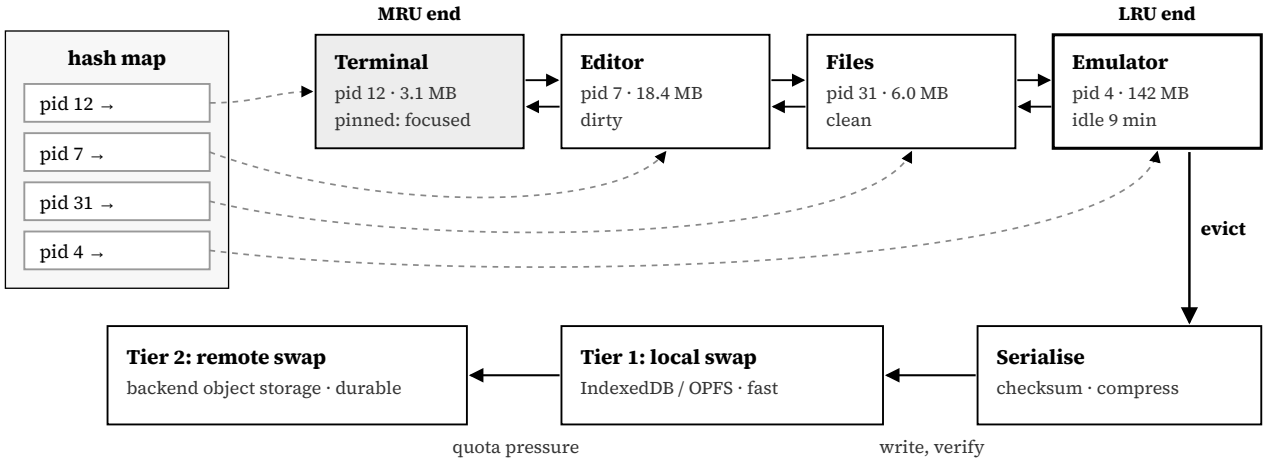


Figure 9.2 — LRU replacement and two-tier swap. The hash map finds any page's list node in O(1), and the list keeps pages in recency order. Eviction takes the least recently used unpinned entry. A page fault reverses the path: the page is read back, its checksum is verified, and it is inserted at the MRU end.

Swap tiers, page faults and thrashing

- **Two-tier swap.** Tier 1 is local swap in IndexedDB or OPFS: fast, but limited by quota. Tier 2 is remote swap in backend object storage, used under local quota pressure (monitor `navigator.storage.estimate()`) and for durability. Clean pages that already have a valid swap copy are dropped without being rewritten.
- **Page faults.** Touching a non-resident page raises a page fault. The process becomes BLOCKED, the page is read and verified, and the process returns to READY. Its window shows a "restoring" state, not a blank.
- **Thrashing guard.** If the page-fault rate stays above a threshold, the OS refuses to start new heavy processes and tells the user, in the spirit of the working-set model.
- **Commands.** `free`, `vmstat` and `pmap <pid>` show the budget, resident and swapped bytes, the fault count and the hit ratio.

ID	Requirement	How it is verified
MEM-01	**Must:** A configurable global memory budget (default 256 MB, measured by your accounting model) and a soft limit for each process.	Settings; `free`.
MEM-02	**Must:** LRU eviction using your own hash map and doubly linked list, with O(1) touch and evict. Focused and pinned processes are never evicted.	Unit tests; code review.
MEM-03	**Must:** Page-out serialises the state, writes it to swap, verifies the checksum and only then releases the memory. Page-in restores a byte-identical state.	Checksum comparison test.
MEM-04	**Must:** Remote swap to backend storage works, and a remotely swapped state can be paged in from a new browser session.	Close the tab, reopen it, restore the state.
MEM-05	**Must:** For a 5 MB state, page-in p95 is ≤ 300 ms from local swap and ≤ 2 s from remote swap.	Benchmark report.
MEM-06	**Must:** With a workload of at least twice the budget open, accounted resident memory never exceeds the budget for more than 1 s, and the tab never crashes.	Stress script at the final defence.
MEM-07	**Must:** Page faults, hit ratio and bytes paged in and out are shown for each process in the Task Manager and `vmstat`.	Live demonstration.
MEM-08	**May:** Memory-mapped files: `sys.mmap(path)` returns a handle, registered in a hash map, whose pages load on first access and are flushed by `msync`.	Credited under DSA and OS.

9.3 The virtual file system

The VFS is a UNIX-like hierarchical file system built as an **n-ary tree of inodes**. Each directory stores its entries in a hash map from name to inode, so looking up a child is O(1) and resolving a path of depth d is O(d). The standard hierarchy is created at first boot (`mkfs`) with the correct owners and modes:

```
PATH              PURPOSE                                   OWNER          MODE
/                 root of the tree                          root:root      rwxr-xr-x
├── bin/          built-in commands and app manifests       root:root      rwxr-xr-x
├── system/       configuration, services, branding, logs   root:root      rwxr-xr-x
│   ├── services/ service manifests (§6.4)
│   └── log/      kernel log, audit log, boot log
├── root/         home of the superuser (uid 0)             root:root      rwx------
├── home/         one home directory per user               root:root      rwxr-xr-x
│   ├── student/  home of uid 1001                          student:users  rwx------
│   └── guest/
├── tmp/          world-writable, sticky bit, cleared on boot root:root    rwxrwxrwt
└── mnt/          optional mount points (shared or cloud)
```

`/root`, `/home`, `/system` and `/bin` are **must**. `/tmp` is **should** and `/mnt` is **may**.

Inodes

Each inode stores the following fields:

- `ino`: inode number;
- `type`: file, directory or symlink;
- `mode`: nine permission bits plus the sticky bit;
- `uid` and `gid`: owner and group;
- `size`;
- `nlink`: link count;
- `atime`, `mtime` and `ctime`: access, modification and change times;
- `blob`: reference to the content, for example a content hash;
- `target`: the target path, for symlinks;
- `version`: a counter used for optimistic concurrency control.

Path resolution

1. Start at `/` for an absolute path, or at the process's working directory for a relative one. Split the path into components and ignore empty ones, so `//` is treated as `/`.
2. For each component, require **execute (search) permission** on the current directory. `.` stays in place, and `..` moves to the parent (at `/` it stays at `/`).
3. Look up the child in the directory's hash map. If it is missing, return `ENOENT`. If a later component follows and the child is not a directory, return `ENOTDIR`.
4. If the child is a symlink, and it is not the last component or the call follows links, resolve its target. Relative targets resolve against the link's own directory. Count the hops and return `ELOOP` after 40.
5. Return the final inode. Operations that must not follow the last link (`lstat`, `unlink`, `readlink`) stop one step early.

Permissions, links and quotas

- **Permissions.** Read, write and execute for owner, group and others. On directories, *r* lists entries, *w* creates and removes entries, and *x* allows traversal. Root (uid 0) bypasses read and write checks. `chmod` accepts octal and symbolic modes, `chown` is root-only, and the default umask is `022`. The sticky bit on `/tmp` means only a file's owner can delete it.
- **Soft links.** `ln -s` stores a path. A link may point to nothing (a dangling link), and `readlink`, `stat` and `lstat` behave as they do on UNIX. Hard links are optional (**may**), but `nlink` must be maintained correctly either way.

- **Hard quotas.** Each user has a byte quota and an inode quota, **enforced inside the same transaction as the write**. A write that would exceed the quota fails with `EDQUOT` and leaves no partial data. `df`, `du` and `quota` report usage. Root is exempt.

Error	Returned when	Error	Returned when
ENOENT	A path component does not exist	**EACCES**	A permission check fails
EEXIST	The target name already exists	**EPERM**	The operation is reserved for the owner or root
ENOTDIR	A directory was expected	**ELOOP**	More than 40 symlink hops
EISDIR	A file operation was applied to a directory	**EDQUOT**	The byte or inode quota would be exceeded
ENOTEMPTY	`rmdir` on a non-empty directory	**EBUSY**	The file is locked by another session
EINVAL	A directory would move into its own subtree	**ENAMETOOLONG**	Name > 255 bytes or path > 4096 bytes

Multi-user synchronisation and distributed file locking

The backend runs a **lock manager** that supports shared (read) and exclusive (write) locks. Locks are **leases**: each has a time-to-live that the holder renews with heartbeats, so a crashed client cannot hold a lock forever. Lock grants and releases are pushed over the WebSocket.

Writes also use **optimistic concurrency**. Every write carries the inode version it expects to replace. If the version does not match, the write is a conflict. Conflicts are never resolved by silently overwriting file content. The losing write is kept as a conflicted copy (*report (conflict from student, 2026-11-14).md*) and the user is notified.

ID	Requirement	How it is verified
VFS-01	**Must:** The VFS is your own n-ary tree of inodes, with hash-mapped directory entries and O(depth) path resolution.	Code review; DSA.md.
VFS-02	**Must:** The standard hierarchy is created at first boot with the specified owners and modes.	`ls -la /`.
VFS-03	**Must:** Path resolution handles absolute and relative paths, `.`, `..`, repeated and trailing slashes, and symlinks, with `ELOOP` after 40 hops.	Path-resolution test suite.
VFS-04	**Must:** Read, write and execute permissions are enforced for owner, group and others on every system call.	Permission matrix test.
VFS-05	**Must:** Soft links can be created, read, followed and left dangling; `lstat` and `stat` differ correctly.	Unit tests.
VFS-06	**Must:** Per-user hard quotas: `EDQUOT` with no partial write; `df`, `du` and `quota` are accurate.	Quota test.
VFS-07	**Must:** Create, read, update and delete, plus `cp -r`, `mv`, `rm -r`, `find`, `stat`, `chmod`, `chown`, `ln -s` and `tree`, from both the terminal and the File Explorer. Both are built on system calls.	Live demonstration.
VFS-08	**Must:** At least 32 levels of nesting; names up to 255 bytes; paths up to 4096 bytes.	Boundary tests.
VFS-09	**Must:** Lease-based distributed locks: a second writer gets `EBUSY` or read-only access, and a crashed holder's lock expires within the lease time.	Two browsers, two users.

ID	Requirement	How it is verified
VFS-10	**Must:** Concurrent changes are detected by version numbers; conflicted copies are kept; no update is ever silently lost.	Concurrency test.
VFS-11	**Should:** `fsck` checks the tree's invariants (every inode reachable, correct link counts, no directory cycles, quota totals consistent) and reports or repairs problems.	Recovery-mode demonstration.

9.4 The window management subsystem

The compositor is the single authority over every window on screen. It keeps a **window registry**, a hash map from window ID to window state:

- the owning process (`pid`), title and icon;
- position and size (`x`, `y`, `w`, `h`) and stacking order `z`;
- the window state (normal, minimised, maximised or full-screen) and the rectangle to restore to;
- whether the window has focus, plus its size limits.

The compositor is responsible for:

- **Focus and z-order.** Exactly one window has focus. Focusing a window raises it. Z-indices are *renumbered* 1…n after each change, so they never grow without limit. Background processes cannot steal focus.
- **Coordinates and transforms.** Dragging and resizing use pointer events with pointer capture. Movement is applied with compositor-friendly transforms (`translate3d`) and batched per animation frame, never by re-laying out the page on every event. The compositor supports resize handles on all eight edges and corners, keeps windows within the viewport, and snaps them to screen edges and halves.
- **Minimise, maximise and restore**, with animations that respect reduced-motion settings. Double-clicking a title bar maximises the window. Keyboard shortcuts are documented.
- **The taskbar registry.** The taskbar shows running applications grouped by app, marks background activity, and lets the user focus or minimise with a click.
- **Window–process coupling.** Closing a window sends a termination signal to its process, and killing a process closes its windows. **Minimised windows are the first candidates for paging** (§9.2), and restoring one triggers a page-in.

ID	Requirement	How it is verified
WM-01	**Must:** The window registry is a hash map with O(1) lookup; the compositor is the only code that writes window geometry.	Code review.
WM-02	**Must:** Dynamic z-ordering with renumbering; click to focus and raise; focus is always visible.	Live demonstration.
WM-03	**Must:** Dragging and resizing with at least 10 windows open keeps the p95 frame time ≤ 16.7 ms.	Performance trace.
WM-04	**Must:** Minimise, maximise, restore, close, snapping and a live taskbar.	Live demonstration.
WM-05	**Must:** Window–process coupling as described above, including paging of minimised windows.	Integration test.
WM-06	**Must:** Window positions, sizes, states and z-order are restored after a reboot.	Reboot test.

ID	Requirement	How it is verified
WM-07	**Should:** System interface meets WCAG 2.2 AA for contrast and keyboard operation, including a keyboard shortcut to switch windows.	Accessibility audit.

9.5 Offline edge execution and synchronisation

Overcoming Puter's cloud coupling (§7.1) needs a **local-first** design. The local VFS store is the primary source for reads and writes. Each change is also appended to an **operation log (outbox)** with an idempotency key and a logical timestamp. When the connection returns, the outbox is replayed to the server in order. The server acknowledges each operation, and conflicts are resolved as described in §9.3.

A Service Worker caches the shell with a versioned cache. New releases install in the background and activate on the next boot, like an OS update with a "restart to update" notice.

ID	Requirement	How it is verified
OFF-01	**Must:** Boot, login, Terminal, File Explorer, Text Editor, Task Manager and Settings work fully offline.	Offline demonstration.
OFF-02	**Must:** Offline changes are queued in a durable outbox and replayed exactly once on reconnection (idempotency keys).	Offline edits, reconnect, compare with server state.
OFF-03	**Must:** Offline login never stores plain-text passwords. Use a salted verifier or a platform authenticator.	Code and storage inspection.
OFF-04	**Must:** The taskbar shows connectivity status and lists degraded services.	Live demonstration.
OFF-05	**Should:** Versioned shell updates install in the background and activate on the next boot.	Deploy a new version during the demo.

9.6 The system-call interface and the terminal

Every application, including your built-in ones, reaches the OS through a typed system-call interface, for example `open`, `read`, `write`, `close`, `stat`, `readdir`, `mkdir`, `unlink`, `rename`, `symlink`, `chmod`, `spawn`, `kill` and `wait`. Calls travel as messages (`postMessage` or `MessageChannel`) in the form `{id, call, args}`, and replies come back as `{id, ok, value | errno}`. **The kernel checks permissions for every call.**

The terminal is a real shell on top of this interface. It supports quoting, pipes (`|`), redirection (`>`, `>>`, `<`), sequencing (`;` and `&&`), background jobs (`&`), environment variables (`$HOME`, `$PATH`, `$USER`), history, and `help` and `man` pages.

Group	Required commands
Files	`ls` `cd` `pwd` `mkdir` `rmdir` `touch` `cat` `echo` `cp` `mv` `rm` `ln` `find` `tree` `stat` `chmod` `chown` `du` `df` `quota` `grep`
Processes	`ps` `top` `kill` `nice` `renice` `jobs` `bg` `fg`
Memory	`free` `vmstat` `pmap`
System	`uname` `whoami` `id` `su` `uptime` `bootlog` `fsck` `lsof` `shutdown` `reboot` `history` `help` `man`
AI (§12)	`ai` (agent), `semsearch` (semantic search), `predict` (pre-fetch model), for the subsystems you implement

ID	Requirement	How it is verified
SYS-01	**Must:** All applications, built-in or not, use only the system-call interface; the kernel checks permissions for every call.	Code review; bypass attempts.
SYS-02	**Must:** The required command set, with pipes, redirection, background jobs and history.	Scripted terminal session.
SYS-03	**Must:** Tab completion of commands, paths and flags is backed by your own trie, with p95 prefix lookup ≤ 1 ms over at least 10,000 entries. The trie is updated incrementally as the VFS changes.	Micro-benchmark; live demonstration.

Section 10

Data Structures and Algorithms Blueprint

Using a library without understanding why is penalised. Each structure below has a mandatory place in your codebase, because each one is the *right* structure for that part of an operating system. You are graded on whether you used the right structure, implemented it correctly, and can prove its complexity.

10.1 Mandatory structures and where they live

Structure	Mandatory location(s)	Required operations and complexity	Why this structure
N-ary tree	VFS directory hierarchy (§9.3)	Resolve O(d); add or remove a child O(1) average; walk a subtree O(k)	It mirrors the hierarchy directly. Moving a directory re-parents one node, whatever the size of its subtree.
Trie (prefix tree)	Terminal autocomplete over commands, paths and history (§9.6)	Insert and remove O(L); complete a prefix O(P + k)	Lookup cost depends on the length of the prefix, not on the number of entries, which gives sub-millisecond completion.
Binary heap (priority queue)	Scheduler ready queue (§9.1); timer and sleep queue	Push and pop O(log n); peek O(1); change priority O(log n) with an index map	The scheduler repeatedly needs the most urgent process, while priorities change as processes age.
Hash maps	Process table; window registry; open-file table; memory-mapped handles; page tables; directory entries	Get, put and delete O(1) average	Every one of these is a lookup by key (PID, window ID, file descriptor or name) on a hot path.
Directed acyclic graph	Service dependencies and boot order (§6.4); application start-up dependencies	Kahn's topological sort O(V + E); cycle detection with the cycle path reported; levels for parallel start	Start-up order is a partial order. Only a DAG expresses it, and a cycle means the system cannot boot.
Hash map + doubly linked list	LRU pager (§9.2); inode cache	Touch, insert and evict O(1)	Neither structure alone gives O(1) recency updates *and* O(1) lookup.
B-tree / hash indexes	Database metadata (§11.4)	Point lookup and range scan O(log n)	They keep metadata queries under 10 ms at scale.

Additional structures earn credit when their use is justified: for example a ring buffer for the kernel log, a Bloom filter in front of the vector index, or an interval structure for window hit-testing.

10.2 Implementation policy

- The **tree, trie, heap, DAG with topological sort, and LRU list must** be written by your team from first principles. Importing a package that provides one of them (a heap or priority-queue library, a trie, a graph library, an LRU cache) for these roles scores **zero** for that item.
- Built-in hash tables (JavaScript `Map` and `Set`, Python `dict`, Go `map`, Rust `HashMap`) are allowed as the hashing primitive. The registries and tables you build on top of them are your own and must be documented. Implementing your own hash table, with a declared collision strategy, for at least one registry is credited as distinction work.
- Each structure lives in its own module under `dsa/`, with unit tests that cover the edge cases (empty structure, one element, duplicates, removal of the root or the head).
- Each structure has a **micro-benchmark** that demonstrates its claimed complexity. For example, autocomplete time should stay flat as the number of entries grows from 10^3 to 10^5.
- `DSA.md` maps every structure to its file path, operations, complexity, and the test and benchmark that prove them.

10.3 Interface sketches

These interfaces are written in TypeScript for illustration only. Your language, naming and details are your own, but your implementation must offer equivalent operations at the stated complexity.

```
interface PriorityQueue<T> {
  push(item: T, priority: number): void;          // O(log n)
  pop(): T | undefined;                             // O(log n), most urgent first
  peek(): T | undefined;                            // O(1)
  changePriority(item: T, priority: number): void; // O(log n) via item → index map
}

interface Trie {
  insert(word: string, meta?: Completion): void;      // O(L)
  remove(word: string): boolean;                      // O(L), prunes empty branches
  complete(prefix: string, limit: number): Completion[]; // O(P + k)
}

interface LRU<K, V> {
  get(key: K): V | undefined;   // O(1), moves the entry to the MRU end
  put(key: K, value: V): void;  // O(1)
  evict(): [K, V] | undefined;  // O(1), least recently used unpinned entry
  pin(key: K): void;  unpin(key: K): void;
}

function topoLevels(services: Manifest[]): Manifest[][];  // Kahn; throws CycleError(path)
```

```
// Scheduler dispatch loop (pseudocode)
on tick():
    while pool.hasIdleWorker() and not ready.isEmpty():
        p ← ready.pop()                                  // best (effectivePriority, seq)
        dispatch(p, pool.idleWorker(), quantumFor(p))
on sliceComplete(p, result):
    if result.done: terminate(p, result.exitCode)
    else if result.blockedOn: block(p, result.blockedOn)
    else: p.seq ← nextSeq(); ready.push(p, p.basePriority)  // back of its level = round-robin
every agingInterval:
    for p in waitingLongerThan(threshold): ready.changePriority(p, max(0, p.effective - 1))
```

Section 11

Database Architecture and Integrity

The database is where your file system becomes durable and shared. Two properties are graded above all others. **Integrity**: bulk operations are atomic, and a failure midway leaves nothing corrupted or orphaned. **Performance**: indexed metadata queries answer in under 10 ms over at least 10,000 records.

11.1 Reference schema

The relational schema below is a reference. A graph database or a document store is acceptable if you defend it (Section 14). For example, a graph model represents the hierarchy and permission inheritance naturally, but you must still prove that updates across many nodes are atomic.

Table	Key columns	Purpose
users	uid PK, username UNIQUE, password_hash, quota_bytes, quota_inodes, used_bytes, used_inodes	Identity and quota accounting.
groups, user_groups	gid PK, name; (uid, gid) PK	Group permissions.
inodes	ino PK, type, mode, uid FK, gid FK, size, nlink, version, blob_id FK, symlink_target, created_at, modified_at, accessed_at	File metadata.
dentries	(parent_ino, name) PK, ino FK	Directory entries. The tree's edges are kept apart from the inodes, which makes hard links possible.
blobs	blob_id PK, sha256 UNIQUE, size, storage_key, refcount	Content, stored by hash so identical files share one copy.
locks	ino PK, mode (S/X), holder_uid, session_id, lease_expires_at	Distributed file locks.
swap_pages	(session_id, pid, page_id) PK, owner_uid, storage_key, size, checksum	Remote swap tier (§9.2).
op_log	op_id PK (idempotency key), uid, session_id, op, payload, client_ts, applied_at	Outbox replay and audit.
file_chunks	chunk_id PK, ino FK, seq, text, content_hash, embedding VECTOR(d)	Semantic search (§12.2).
telemetry_events	event_id PK, uid, app_id, kind, ts, context	Pre-fetch model training data (§12.4), when synced by opt-in.

Table	Key columns	Purpose
audit_log	id PK, uid, actor (user/agent), syscall, args, result, prompt_id, ts	Accountability for every AI action.
sessions	session_id PK, uid, window_layout, updated_at	Session restore after reboot.

11.2 Storing a tree in a database

How you store the hierarchy decides how expensive your bulk operations are. Choose a representation, justify it against your workload, and prove that your bulk operations stay atomic with the one you choose.

Representation	Move / rename a directory	Read a subtree	Verdict for a file system
Adjacency list (parent_ino)	One row updated, regardless of subtree size	Recursive query (e.g. `WITH RECURSIVE`)	Strong default: cheap writes, and moves are trivially atomic.
Materialised path	Every descendant's path rewritten, in bulk	Fast prefix scan on an index	Fast reads, but every move is a bulk write that must be atomic.
Closure table	O(subtree × depth) rows deleted and inserted	Simple join, fast	Good for reads and ancestor checks, heavy for moves.
Nested sets	Large parts of the tree renumbered	Very fast range scan	Unsuitable for a file system with frequent writes.

11.3 ACID guarantees for bulk operations

These operations **must** be atomic: recursive delete (`rm -r`), recursive copy (`cp -r`), directory moves (especially to a new parent), batch renames (for example, 500 files renamed by the agent), recursive `chmod` and `chown`, and the quota updates that go with all of them. Seven rules apply:

1. **One transaction per user-visible operation.** The whole subtree changes, or none of it does.
2. **Quota checked inside the transaction.** Lock the user's quota row (`SELECT … FOR UPDATE` or equivalent) so that two concurrent writes cannot both pass the check.
3. **Invariants checked before commit.** A directory cannot move into its own subtree (`EINVAL`). Names are unique within a directory, enforced by a unique index rather than application code.
4. **Any error rolls back.** The client receives one clear error code, and the database is exactly as it was before.
5. **The client mirror is also atomic.** The local change is one IndexedDB transaction across every affected object store. A half-applied local state is never committed.
6. **Retries are safe.** Idempotency keys make sure a retried operation is applied at most once.
7. **The isolation level is declared and justified.** For example: SERIALIZABLE with automatic retry on serialisation failure, or READ COMMITTED with explicit row locks. Explain why your choice is safe for tree operations.

```
-- Directory move with an adjacency list: one row changes, atomically
BEGIN ISOLATION LEVEL SERIALIZABLE;
  -- 1. lock source and destination entries; check write permission on both parents
  -- 2. refuse if :dest is inside the subtree of :src (recursive ancestor check) → EINVAL
  UPDATE dentries SET parent_ino = :dest, name = :new_name
   WHERE parent_ino = :src_parent AND name = :src_name;
  INSERT INTO op_log (op_id, uid, op, payload) VALUES (:idempotency_key, :uid, 'mv', :payload);
COMMIT;   -- any error ⇒ ROLLBACK: nothing changed

-- Integrity probes your fault-injection suite runs after every failed operation
SELECT i.ino FROM inodes i LEFT JOIN dentries d ON d.ino = i.ino
 WHERE d.ino IS NULL AND i.ino <> 1;                        -- orphaned inodes: must be empty
SELECT d.* FROM dentries d LEFT JOIN inodes p ON p.ino = d.parent_ino
 WHERE p.ino IS NULL;                                         -- entries with no parent: must be
empty
```

Mandatory fault-injection harness

Your test build **must** include a hook that throws an error after *N* of *M* node operations. The reference test runs `rm -r` on a 1,000-node tree and fails at node 500. It then checks that the tree's checksum is unchanged, that both orphan probes return nothing, and that the user's quota usage is unchanged. The same test runs for `mv`, `cp -r`, batch rename and `chmod -R`. It is repeated live at the final defence.

11.4 Index optimisation

- **A unique B-tree index on `dentries(parent_ino, name)`.** It serves every step of path resolution and every directory listing, and it enforces unique names. Mirror it on the client with a compound IndexedDB index on [parent, name].
- **An index on `inodes(uid, modified_at)`** for "my recent files" and quota reports, and a **hash index on `blobs(sha256)`** for de-duplication.
- **A name-search index**: a B-tree on the lower-cased name with a pattern operator class for prefix search, and optionally a trigram index for substring search.
- **A vector index** (HNSW or IVF) on embeddings, once exact search no longer meets the latency target (§12.2).

Query	Benchmark query (run against ≥ 10,000 inodes; 100,000 recommended)	Target (p95)
Q1	Resolve a path of depth 8	≤ 10 ms
Q2	List a directory with 500 entries, sorted by name	≤ 10 ms
Q3	Prefix search on file name across one user's files	≤ 10 ms
Q4	A user's files modified in the last 7 days, newest first, limited to 50	≤ 10 ms
Q5	Total size of a 1,000-node subtree (`du`), using a maintained aggregate or a justified recursive query	≤ 10 ms

The benchmark report **must** include a reproducible seed script (at least 50 users, with a realistic spread of directory depths), p50 and p95 over 1,000 runs *before and after* indexing, and query plans (`EXPLAIN ANALYZE` or equivalent) proving that the indexes are used.

11.5 Database requirements

ID	Requirement	How it is verified
DB-01	**Must:** A documented schema (ERD) with normalisation reasoning; versioned migrations, each with a rollback script; a documented backup-and-restore procedure.	DATABASE.md; migrations run from scratch.
DB-02	**Must:** Every bulk VFS operation is atomic. The fault-injection suite passes with zero orphans and unchanged checksums.	CI and live test.
DB-03	**Must:** A declared isolation level with retry logic. Two clients moving overlapping subtrees at the same time always leave a consistent tree.	Concurrency test.
DB-04	**Must:** Constraints are enforced by the database: foreign keys, unique (parent, name), checks on type and mode.	Schema review; constraint-violation tests.
DB-05	**Must:** Indexes as described in §11.4; benchmark queries Q1–Q5 meet p95 ≤ 10 ms at ≥ 10,000 records, with query plans as evidence.	Benchmark report; live query.

Section 12

Foundational AI Integration

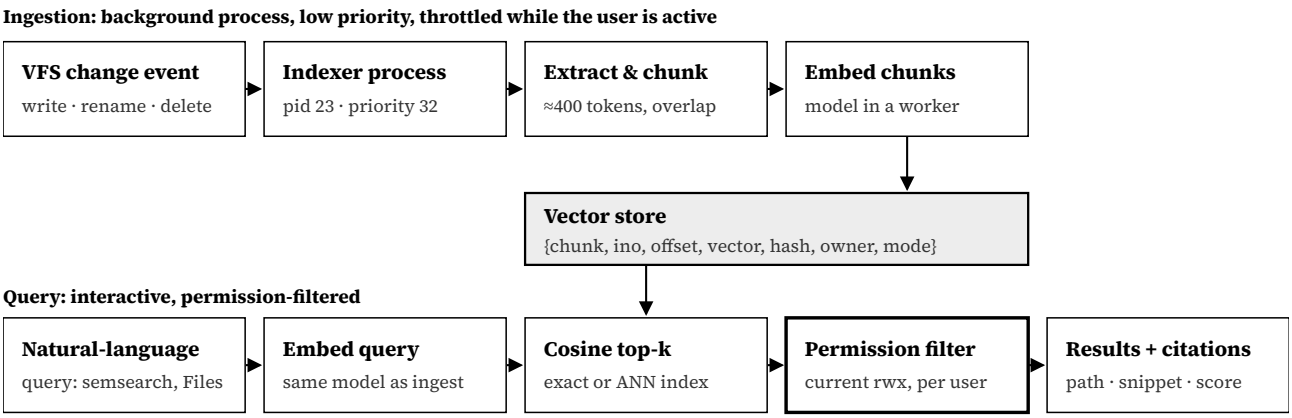
AI cannot be a cosmetic chatbot. It must be a system-level subsystem connected to OS primitives. Each team **must** implement **at least two** of the three subsystems in this section. Depth and coupling are graded, not the count: two subsystems built deeply outrank three built superficially.

12.1 Choosing your subsystems

Subsystem	What it adds to the OS	Primary OS primitives it must touch
A. Semantic VFS search (RAG)	Find files by meaning, not only by name, and answer questions about them with citations.	VFS change events, scheduler (background indexing), permissions, database.
B. Agentic system execution	Turn natural-language instructions into validated, atomic sequences of system calls.	Shell, system-call interface, permissions, transactions, audit log.
C. Predictive pre-fetching	Learn usage patterns locally and load applications before they are opened.	Scheduler (idle-priority jobs), memory manager, telemetry.

12.2 Subsystem A: semantic VFS search (RAG pipeline)

File contents are embedded into a vector store. Users query their file system with natural-language concepts, for example *“find the document where I discussed the memory leak in the UI component”*, and results are ranked by cosine similarity.



similarity(q, d) = (q · d) / (‖q‖ ‖d‖) with unit-length vectors this is simply the dot product q · d

Figure 12.1 — Ingestion and query pipelines. The permission filter uses the *current* permissions at query time. A user must never see a snippet from a file they cannot read, even if the index was built before the permissions changed.

- **Ingestion.** The VFS emits change events. A background indexer process extracts text (plain text, Markdown and source code at minimum), splits it into overlapping chunks that respect paragraph and heading boundaries, embeds each chunk, and stores it with its inode, offset, content hash and owner.
- **Incremental updates.** The content hash lets the indexer skip unchanged chunks. A rename updates path metadata without re-embedding. A delete removes the file's vectors.
- **Model.** A local embedding model running in a worker (for example a small sentence-embedding model through a WebAssembly or WebGPU runtime), or a hosted embedding API called through your backend. Justify the choice on cost, latency, privacy and offline behaviour.
- **Index.** Exact brute-force search is acceptable while it meets the latency target; prove this with measurements. Beyond that, use an approximate index (HNSW, IVF) or a vector database such as pgvector or Qdrant.
- **Answering.** Optionally, the top chunks go to a language model that writes an answer citing `path:line` for every claim.
- **Evaluation.** Build a corpus of at least 200 documents and at least 30 natural-language queries with known relevant files. Report Recall@5 and mean reciprocal rank (MRR). The examiners hold a hidden query set for the final defence.

ID	Requirement	How it is verified
RAG-01	**Must:** The indexer is a scheduled background process: visible in `ps`, low priority, throttled while the user is active.	`ps`; `top` during typing.
RAG-02	**Must:** The index stays consistent with the VFS: create, modify, rename and delete are reflected within 30 s for a 50 KB text file.	Timed test.
RAG-03	**Must:** No result ever includes content the querying user cannot read.	Permission test suite: zero leaks allowed.
RAG-04	**Must:** Recall@5 ≥ 0.80 on the team's evaluation set, with MRR reported.	Evaluation report; hidden set.
RAG-05	**Must:** Retrieval latency p95 ≤ 500 ms over ≥ 10,000 chunks, not counting optional answer generation.	Benchmark.

ID	Requirement	How it is verified
RAG-06	**Must:** Integrated into the shell (`semsearch`) and the File Explorer. When the model is unavailable, search falls back to keyword search with a clear notice.	Live and offline demonstration.

12.3 Subsystem B: agentic system execution

An LLM agent built into the shell turns natural-language prompts into valid OS operations. It resolves arguments and runs complex file-manipulation pipelines **safely**. The model only ever *proposes*; the kernel validates, previews, asks for confirmation, runs and records.

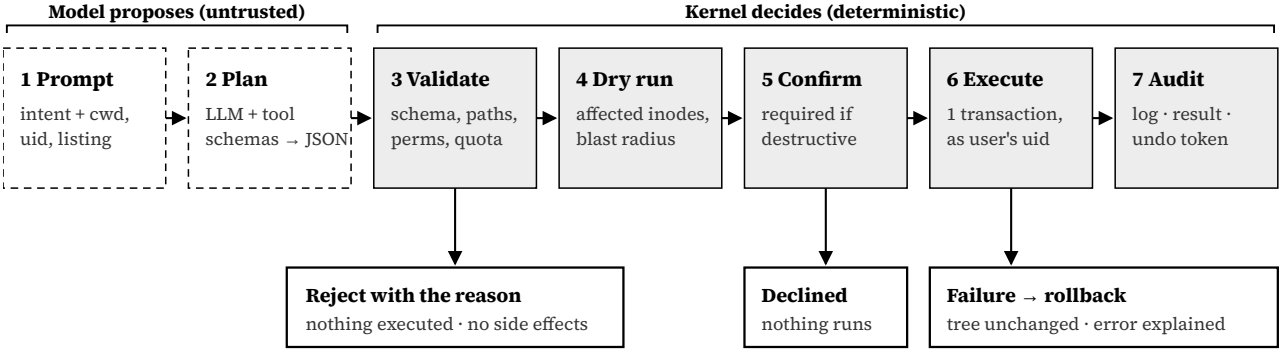


Figure 12.2 — The agent pipeline. Steps 3–7 are ordinary, deterministic kernel code. The model's output is data that must pass validation. It is never executed as text.

```
student@your-os:~$ ai "move every PDF I changed last week into a new folder called reports, and
make it read-only for others"
Plan · 4 steps · 1 transaction
  1  mkdir   ~/reports                                    create 1 dir
  2  find    ~ -type f -name "*.pdf" -mtime -7            → 12 files
  3  mv      <12 files> ~/reports/                        modify 12 entries
  4  chmod   -R o-w ~/reports                             modify 13 inodes
Blast radius: 13 inodes · 0 deletions · 0 overwrites · within quota
Proceed? [y/N/e=edit] y
✓ committed in 41 ms · txn 8f2c · undo with: ai undo 8f2c
```

ID	Requirement	How it is verified
AGT-01	**Must:** Tools are typed system calls with JSON schemas. Model output is parsed as data and never passed to `eval` or run as a shell string.	Code review.
AGT-02	**Must:** Invalid plans (unknown command, bad arguments, missing path, permission denied, quota exceeded) are rejected *before* anything runs, with the reason. Zero partial executions.	Invalid-plan test set.
AGT-03	**Must:** Destructive operations (delete, overwrite, `chmod` or `chown`, or any plan touching more than 20 inodes) need explicit confirmation after the dry run. Zero bypasses.	Destructive-prompt test set.
AGT-04	**Must:** The agent runs with the invoking user's privileges. It cannot escalate, and it cannot modify `/system` or `/bin` unless the user is root and confirms.	Privilege tests.
AGT-05	**Must:** A multi-step plan runs atomically, as one transaction or with complete compensating rollback.	Fault injection during a plan.

ID	Requirement	How it is verified
AGT-06	**Must:** Prompt-injection resistance: file contents and names enter the model's context marked as untrusted data. An instruction hidden in a file (fixture supplied by the examiners) must cause no tool call the user did not ask for.	Injection fixtures.
AGT-07	**Must:** ≥ 16 of 20 instructions from the examiners' benchmark are completed correctly. The benchmark includes multi-step pipelines, relative dates, glob patterns and ambiguous requests that need a clarifying question.	Hidden benchmark at the final defence.
AGT-08	**Must:** Every agent system call is written to the audit log with the prompt, plan, confirmation and result, and can be viewed with `ai log`.	Audit inspection.
AGT-09	**Must:** The agent is a real process: visible in `ps`, and killable in the middle of a plan, which rolls the transaction back.	Kill during execution.
AGT-10	**May:** `ai undo <txn>` reverses a committed plan using the operation log.	Credited under AI depth.

12.4 Subsystem C: predictive process pre-fetching

A local machine-learning model analyses the user's telemetry to predict which applications they will use next. It loads their assets into memory before they are opened, so that launch feels instant.

- **Telemetry.** Application launch, focus and close events, with time of day, day of week, the previous application and the position in the session. Events are stored locally in a bounded ring buffer. The user can view and clear them in *Settings → Privacy*. Nothing leaves the device without opt-in.
- **Model.** It must train and run *locally*, inside a worker. Acceptable choices include a first- or second-order Markov chain over launch sequences conditioned on time of day, naive Bayes, a small decision tree or logistic regression, k-nearest neighbours over context vectors, or a tiny neural network. Justify any model more complex than a Markov chain by showing that it beats one.
- **Pre-fetch policy.** When a prediction's confidence passes a threshold *and* the system is idle *and* there is free memory, the scheduler starts a pre-fetch process at the lowest priority. It loads the application's code, warms its caches and pages in its last state. Pre-fetched pages are marked *speculative* and are the first to be evicted.
- **Learning.** The model updates incrementally after each launch. Until it has enough data, it falls back to most-frequently-used.
- **Evaluation.** Replay a recorded trace of at least 500 launch events, split chronologically into training and test sets so the model cannot see the future. Report top-1 and top-3 accuracy against the most-frequently-used (MFU) and most-recently-used (MRU) baselines, launch latency with and without pre-fetch, and wasted pre-fetch bytes.

ID	Requirement	How it is verified
PRE-01	**Must:** The model trains and runs locally in a worker; one prediction takes ≤ 5 ms.	Benchmark.
PRE-02	**Must:** Top-3 accuracy beats the better of the MFU and MRU baselines on the held-out trace.	Evaluation report.
PRE-03	**Must:** For correctly predicted launches, pre-fetching cuts median launch latency by ≥ 30%.	Latency comparison.

ID	Requirement	How it is verified
PRE-04	**Must:** Pre-fetching never slows the foreground: it runs only when idle, at the lowest priority, within free memory, and its pages are evicted first.	Frame-time trace during pre-fetch.
PRE-05	**Must:** Telemetry stays on the device by default and can be viewed and cleared; syncing it requires opt-in.	Settings demonstration.
PRE-06	**Must:** Predictions are visible: `predict` and the Task Manager show current predictions with confidence, and pre-fetch processes appear in `ps`.	Live demonstration.

12.5 The coupling map

Include this table, completed for your own system, in `AI.md`. Examiners use it to judge whether your AI is foundational.

	Scheduler	Memory	VFS	Shell	Database	Security
A. RAG	Indexer is a niced background process	Model and index pages can be evicted	Subscribes to change events; reads via system calls	`semsearch`; Files search	`file_chunks` with vector index	Permission filter at query time
B. Agent	Agent is a killable process	—	Acts only through system calls	`ai` command	Plan runs as one transaction; audit log	User's uid; confirmation; injection defence
C. Pre-fetch	Emits idle-priority pre-fetch jobs	Pages state in; speculative pages evicted first	Warms recently used files	`predict`	Optional `telemetry_events`	Local-only telemetry by default

12.6 Models, cost and offline behaviour

- **Choice of model.** Hosted APIs or local models (on your backend, or in the browser through WebAssembly or WebGPU runtimes) are both acceptable. Justify the choice on quality, latency, cost, privacy and offline behaviour.
- **Keys stay on the server.** API keys never appear in the client bundle or the repository. Every model call goes through your backend, with per-user rate limits.
- **Cost accounting.** Log tokens and cost for every request, set a budget cap, and report the total cost of development and the demonstration.
- **Graceful degradation.** Offline or when the model fails: semantic search falls back to keyword search, the agent falls back to the plain terminal with an explanation, and a local pre-fetch model is unaffected.
- **Reproducible tests.** Benchmarks use fixed seeds and temperature 0. CI uses recorded model responses so that tests do not depend on a live model.

Part IV

Delivery and Assessment

How the work is delivered, defended and graded: the phase plan and its gates, the written architecture defence, the engineering standards your repository must meet, the rubric, the protocol for the final defence, and the rules on integrity and AI assistance.

Section 13

Milestones and Deliverables

The project is delivered in five phases. Each phase ends with a **gate**: a live review in which the phase's criteria must be demonstrated on the tagged release. Weeks below are indicative. Exact dates are announced on Google Classroom.

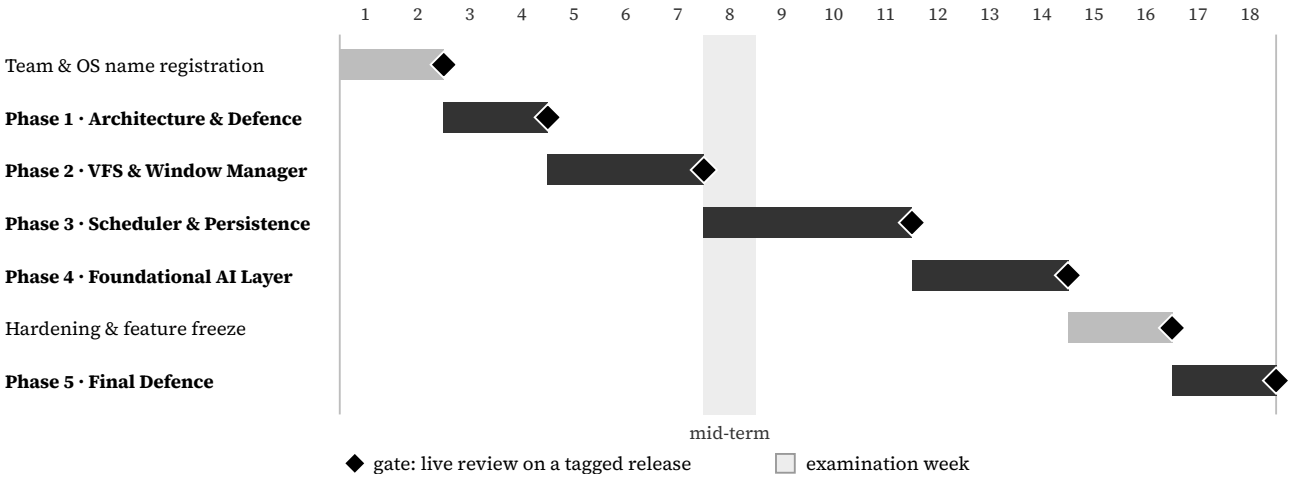


Figure 13.1 — The 18-week plan. Each phase is a vertical slice of working software, not a stack of documents. Phase 3 spans the mid-term examination week, so plan your capacity for it.

13.1 Phase deliverables and gate criteria

Phase and week	Deliverables	Gate: must be demonstrable
Registration Week 2	Team registration form (Appendix D); OS name and tagline; roles and secondary reviewers; `TEAM_CONTRACT.md`; repository created with a CI skeleton; instructor added as a collaborator.	Five members confirmed; name approved as unique and appropriate; repository reachable; CI runs on a pull request.
Phase 1 Architecture & Defence Week 4	System architecture diagrams (context, containers, components); formal technology-stack justification (`STACK_DECISION.md`, Section 14); database schema (ERD, DDL and chosen tree representation); DSA blueprint mapping (`DSA.md` draft); boot design (service list, DAG, critical and degradable services); two-page benchmark analysis of Puter and daedalOS; work breakdown with an owner for every task; risk register.	15-minute architecture defence plus 10 minutes of questions. Every member presents. The stack is approved or returned for revision within one week.

Phase and week	Deliverables	Gate: must be demonstrable
Phase 2 VFS & Window Manager Week 7 · v0.2.0	Boot screen v1 with identity kit, truthful progress for asset loading and kernel initialisation, and a login screen; a working desktop; the multi-window compositor (WM-01 to WM-06); the tree-based VFS with full create, read, update and delete, path resolution, permissions, soft links and quotas (VFS-01 to VFS-08); terminal with file commands and trie autocomplete (SYS-03); File Explorer; public deployment.	VFS test suite green; 10 windows dragged at 60 FPS; the permission and quota tests run live; the deployed URL boots.
Phase 3 Scheduler & Persistence Week 11 · v0.3.0	Process table, heap scheduler, Web Worker pool, process commands and Task Manager (PROC-*); LRU paging with local and remote swap (MEM-*); atomic database persistence with the fault-injection suite, indexes and benchmark (DB-*); distributed locks and multi-user sync (VFS-09, VFS-10); offline execution (OFF-*); boot v2: DAG-driven, verbose log, boot menu, degraded and offline boot (BOOT-*).	Frame time held under CPU load; paging shown with a 2× workload; a fault-injected `rm -r` rolls back live; offline boot; two users contend for a lock.
Phase 4 Foundational AI Layer Week 14 · v0.4.0	At least two AI subsystems integrated (RAG-*, AGT-*, PRE-*); evaluation reports; AI safety results (permission leaks, injection fixtures, destructive confirmations); `AI.md` with the coupling map; model cost report.	Live AI demonstration on examiner prompts; the litmus test (§3.4); the safety tests run live.
Feature freeze Week 16 · v1.0.0-rc	Release candidate tagged; all documentation complete; a full rehearsal of the final defence.	Rehearsal passed. After the freeze, only bug fixes are merged.
Phase 5 Final Defence & Codebase Weeks 17–18 · v1.0.0	Production deployment; comprehensive technical report; test suite with coverage report; end-to-end system demonstration; demonstration video; individual viva (Section 17).	The Demo Day protocol.

13.2 Gate rules

- A gate is passed only by a **live demonstration on the tagged release**. Slides, screenshots and recorded videos do not pass a gate.
- A missed gate leads to a mandatory meeting with the instructor and a written recovery plan within one week.
- If a gate is still unmet at the next phase, the project mark is capped at 80%. If two gates are unmet, it is capped at 70%.
- Features merged after the feature freeze are reverted before judging.
- A member who is absent from the final defence without prior approval scores zero for the project.

Section 14

The Architecture Defence

Because the stack is open, your reasoning is part of what is graded. The technology-stack justification carries 10% of the marks directly, and it affects every other criterion, because a weak choice made in Week 4 becomes a weak system in Week 18.

14.1 Structure of `STACK_DECISION.md`

Write one decision record for each major decision, and keep them all in `STACK_DECISION.md`. Each record contains:

1. **Context and constraints.** The WebOS constraints this decision affects, ranked. For example: main-thread responsiveness, offline operation, atomicity of tree operations, multi-user consistency, cost, team skills.
2. **Options.** At least three genuine candidates, including one you expected to lose.
3. **Criteria and weights, declared before scoring.** A weighted decision matrix whose criteria are not reverse-engineered from the winner.
4. **Evidence.** A prototype, a measurement or a cited benchmark for each claim that affects the outcome. Opinions are not evidence.
5. **Decision and consequences.** What you chose, what you gave up, and the risks you accept, with how you will mitigate them.
6. **Reversal trigger.** The observation that would make you revisit the decision, for example “if lock-grant latency exceeds 200 ms under 50 clients”.

14.2 Decisions that must be defended

Decision	Typical options	What your defence must address
UI framework	React, Svelte, Vue, Solid, plain Web Components	The rendering cost of many windows; isolating updates so one window's change does not re-render others; bundle size against the boot budget.
Kernel and DSA language	TypeScript; Rust or Go compiled to WebAssembly	Raw performance against the cost of crossing the JavaScript–WebAssembly boundary; debugging; team skills.
Backend runtime	Node.js, Deno, Bun, Go, Rust, Python	The concurrency model for thousands of long-lived WebSocket connections; the library ecosystem; memory per connection.
Real-time transport	WebSockets, Server-Sent Events, long polling, short polling, WebRTC data channels	Two-way process and lock events; latency; idle cost; reconnection and message ordering.
Primary database	PostgreSQL, MySQL, SQLite, MongoDB, a graph database	ACID guarantees for bulk tree operations; recursive queries; indexing; how permissions are modelled.
Client storage	IndexedDB, OPFS, Cache API	Transactions against raw throughput; quota and eviction behaviour; use from workers.
Vector store	pgvector, Qdrant, Chroma, your own index	Scale and latency; keeping vectors next to metadata so permissions can be filtered.

Decision	Typical options	What your defence must address
AI models	Hosted or local; choice of embedding model and LLM	Quality, cost per request, latency, privacy, offline behaviour.
Deployment	Docker Compose on a VPS, a platform service, managed databases	WebSocket support, cost, reproducibility, and bring-up in a few commands.

14.3 What a strong justification looks like

Weak (earns little or nothing)

"We chose WebSockets because they are fast and modern, and we chose MongoDB because it is flexible and popular."

Strong (the standard expected)

Process-state changes, lock grants and sync acknowledgements are initiated by the server, and arrive at up to about 20 per second per client during bulk operations.

Polling every second would add up to a second of latency to every lock grant and cost about 3,600 requests per hour per idle client. Long polling reduces the latency but still opens a new request for every event.

A WebSocket gives one full-duplex connection with server push. Our prototype measured a median lock-grant latency of 38 ms, against 510 ms with one-second polling.

Risk: some networks drop idle sockets. We mitigate this with 25-second heartbeats and resumable sessions using sequence numbers. We will revisit the decision if we need to support clients behind proxies that block WebSocket upgrades.

The figures here are illustrative. Yours must come from your own measurements.

14.4 The defence session

The Phase 1 defence is 15 minutes of presentation and 10 minutes of questions. Every member presents part of the architecture, and the examiners may ask *any* member about *any* decision. A decision that only one member can explain is treated as undefended. If the defence is not accepted, the team revises and re-presents within one week. The stack is then frozen; changing it later needs a new decision record and the instructor's approval.

Section 15
Repository, Documentation and Testing Standards

15.1 Repository layout

The layout below is recommended. You may adapt it, but the subsystem boundaries must be visible in the directory structure.

```
<your-os-name>/
├── apps/        built-in applications: files, terminal, editor, task manager, settings
├── kernel/      process table, scheduler, worker pool, memory manager, system calls, IPC
├── vfs/         inode tree, path resolver, permissions, quotas, lock client, sync client
├── wm/          compositor, taskbar, boot, login and desktop screens
├── ai/          indexer, vector search, agent, predictor, eval/ (datasets and harnesses)
├── dsa/         heap, trie, tree, dag, lru, each with tests/ and bench/
├── services/    service manifests and the service manager (boot DAG)
├── server/      API, sync engine, lock manager, swap storage, LLM gateway
├── db/          migrations (up and down), seed scripts, benchmark queries
├── tests/       unit, integration, e2e, performance, fault-injection
├── docs/        diagrams, decision records, identity kit
└── README.md  ARCHITECTURE.md  STACK_DECISION.md  DSA.md  DATABASE.md  AI.md
    BOOT.md  TESTING.md  SECURITY.md  AI_USAGE.md  TEAM_CONTRACT.md  CHANGELOG.md
```

15.2 Required documents

Document	Contents
README.md	What the system is, its name and tagline, the live URL, bring-up in three commands or fewer, what works, and known limitations stated honestly.
ARCHITECTURE.md	Diagrams, module boundaries, the system-call protocol, data flows, and the two decisions you would reverse with hindsight.
STACK_DECISION.md	The decision records of Section 14.
DSA.md	Every mandatory structure: location, operations, complexity, proving test and benchmark.
DATABASE.md	ERD, tree representation, transaction design, isolation level, indexes, benchmark results with query plans.
AI.md	Subsystems chosen, models and why, the coupling map (§12.5), evaluation results, safety controls, cost report.
BOOT.md	Boot stages, the service DAG, critical and degradable services, boot-time budget and the measured critical path.
TESTING.md	Test strategy, how to run each suite, coverage, performance budgets, fault-injection design.
SECURITY.md	Threat model, permission model, agent trust boundary, prompt-injection defences, secret handling.
AI_USAGE.md	Disclosure of AI coding assistance (Section 18).

15.3 Testing strategy

Level	What it must cover	Runs
Unit	Every DSA structure with its edge cases; the path resolver; the permission matrix; the shell parser; the scheduler's ordering and aging.	Every pull request
Property-based	**Should.** Random sequences of VFS operations always leave `fsck` clean; heap invariants hold after random pushes, pops and priority changes.	Every pull request
Integration	System calls against the database; fault injection for every bulk operation; concurrent clients; lock leases; outbox replay.	Every pull request

Level	What it must cover	Runs
End-to-end	Browser automation: boot → login → open the terminal → file operations → reboot → session restored; the offline boot path.	Main branch, and each release
Performance	Boot time, frame time under load, query p95, autocomplete latency, page-in latency, each with a budget that fails the build when exceeded.	Nightly, and each release
AI evaluation	Recall@5 and MRR, the agent benchmark, injection fixtures, prediction accuracy, using recorded model responses in CI.	Each release

Coverage targets: at least 70% line coverage overall and at least 85% on `dsa/`, `vfs/` and `kernel/`. Coverage is reported by CI on every pull request. A high coverage number from shallow tests is worth less than a smaller number from tests that check behaviour; examiners read the tests.

15.4 Commits and collaboration

- Small, focused commits with meaningful messages (a convention such as `feat:`, `fix:`, `test:`, `docs:` is recommended). Reference requirement IDs where relevant.
- All work arrives through pull requests that are reviewed by another member. Review comments are part of the evidence of engineering process.
- The main branch is protected: no force pushes, and CI must pass before merge.
- Work is tracked on an issue board with an owner for every issue.
- Commit authorship must be genuine. Pair-programmed work is credited with co-author trailers. Rewriting history to misrepresent contributions is misconduct.

Section 16

Evaluation Rubric

16.1 Grading criteria and weights

Criterion	Weight	What is assessed	Primary evidence
OS & Process Engineering	25%	Robustness of Web Worker scheduling, efficiency of memory paging, UNIX-like VFS behaviour, window management, the boot sequence and offline execution.	BOOT, PROC, MEM, VFS, WM, OFF and SYS tests; live stress tests.
DSA Rigour	20%	Explicit, optimal use of trees, tries, heaps, DAGs, hash structures and the LRU, written by the team without leaning on abstraction libraries.	DSA.md; unit tests; benchmarks; code review.
AI Integration Depth	20%	How deeply the AI is coupled to the system. Does it act as an integral OS controller or a superficial add-on?	RAG, AGT and PRE tests; coupling map; litmus test; hidden benchmarks.
Database Architecture	15%	Schema scalability, strict ACID guarantees during bulk operations, and indexing strategy.	DB tests; fault-injection results; query plans; benchmark report.

Criterion	Weight	What is assessed	Primary evidence
Tech Stack Justification	10%	The technical reasoning behind the chosen languages, runtimes and distributed architecture.	STACK_DECISION.md; Phase 1 defence; viva answers.
Code Quality & Equity	10%	Commit history, clean architecture and naming, and verifiable equal distribution of work across all five members.	Repository history; pull-request reviews; CI; peer assessment.
Total	**100%**		

16.2 Performance levels

Criterion	Exemplary (90–100%)	Proficient (70–89%)	Developing (50–69%)	Beginning (below 50%)
OS & Process (25%)	Scheduler with aging and a worker pool keeps p95 ≤ 16.7 ms under load. LRU paging with both swap tiers restores byte-identical state. Full UNIX-like VFS semantics, including links, quotas and locks. Truthful DAG-driven boot with safe mode, recovery and offline boot.	All primitives work, with minor gaps: e.g. aging bound not proven, slow remote swap, limited recovery options.	Scheduler exists but some heavy work runs on the main thread. Paging is partial or manual. VFS lacks permissions or links. Boot partly simulated.	Process table is cosmetic. No paging. Flat or `localStorage` file storage. Timer-driven boot screen.
DSA (20%)	Every mandatory structure written by the team, optimal, tested and benchmarked, with complexity shown empirically, and used where the problem demands it.	All present and correct, with tests; benchmarks limited.	One or two replaced by libraries or sub-optimal (e.g. a sorted array as the queue, linear-scan autocomplete).	Structures missing, or present but not actually used by the system.
AI depth (20%)	At least two subsystems deeply coupled to the scheduler, VFS and shell. Evaluation targets met. Robust safety: confirmation, atomic plans, injection resistance, permission-safe retrieval. Graceful offline behaviour.	Two subsystems working and integrated with at least the shell and VFS; most targets met; safety controls present.	One solid subsystem and one shallow one; AI bypasses system calls or permission checks in places.	A chatbot widget; AI not connected to OS primitives.

Criterion	Exemplary (90–100%)	Proficient (70–89%)	Developing (50–69%)	Beginning (below 50%)
Database (15%)	Well-justified, normalised schema. All bulk operations atomic under fault injection and concurrency. Indexes proven with query plans; p95 ≤ 10 ms at 100,000 records.	Atomic bulk operations; targets met at 10,000 records; minor schema weaknesses.	Some bulk operations non-atomic or untested; indexing ad hoc; targets missed.	No transactions; orphans possible; no indexing strategy.
Stack (10%)	Every major decision has criteria set in advance, at least three alternatives, measured evidence, honest risks and reversal triggers, and every member can defend it.	Decisions justified against criteria; evidence limited.	Justification mostly general statements with little evidence.	Popularity or familiarity only; reasoning worked backwards from the choice.
Quality & equity (10%)	Clean modular code, consistent naming, reviewed pull requests, CI green, coverage met, and contributions evenly spread and verifiable across all five members.	Mostly clean; reviews usually present; contributions reasonably balanced.	Inconsistent quality; large unreviewed commits; one or two members dominate.	Large unreviewed code dumps; one member wrote most of the code; no tests.

16.3 Where the boot screen and identity are assessed

The boot sequence (Section 6) is assessed under **OS & Process Engineering**, and its service DAG also counts under **DSA Rigour**. The OS name and identity kit (Section 5) are *entry requirements*: a submission without a registered name, or without a working boot screen, does not pass the Phase 2 gate.

16.4 Individual moderation

The team mark is the starting point for each member. It is moderated individually using three sources of evidence:

- **viva performance** across the whole system;
- **contribution evidence** from commits, pull requests and reviews;
- **confidential peer assessment**, collected twice in the semester.

A member who cannot explain code they submitted, or the parts of the system they are asked about, receives a reduced mark whatever the team's result. A member with strong evidence of carrying the team is recognised as well.

Section 17

The Final Defence

Demo Day tests the system, not the slides. Each team has 30 minutes, run to the protocol below on a machine the team does not control.

17.1 Demo Day protocol

Time	Segment	What happens
0:00–0:05	**Clean bring-up**	An examiner clones the repository and starts the system with your documented commands (three or fewer, finishing within five minutes), then opens the production URL. This is scored before you speak.
0:05–0:08	**Boot**	Cold boot in verbose mode; `bootlog --critical-path`; the network is disconnected and the system rebooted to show offline boot and the core utilities.
0:08–0:18	**System under test**	Examiner-driven checks against Appendix A: • a CPU-bound process while windows are dragged, with a frame meter visible; • a workload of twice the memory budget, with paging shown in the Task Manager; • a fault-injected recursive delete that rolls back; • two users contending for a lock; • hidden AI prompts: a semantic query, an agent instruction including a destructive confirmation, and a pre-fetch demonstration.
0:18–0:26	**Individual viva**	Each of the five members is questioned on a subsystem chosen by the examiners, not necessarily their own.
0:26–0:30	**Judges' questions**	Architecture, trade-offs, limitations, and what you would do differently.

17.2 Final deliverables

- The repository tagged `v1.0.0`, with every document from §15.2 up to date with the code.
- A production deployment reachable at a public URL, with a demo account for examiners.
- A test and coverage report generated by CI for the tagged release.
- A 5–7 minute demonstration video that makes sense without narration.
- The slides used on Demo Day.
- A **technical report** of 20–30 pages, containing:
  1. introduction and motivation;
  2. the OS identity and its distinctive idea;
  3. architecture;
  4. a chapter for each subsystem (boot, processes and scheduling, memory, VFS, window manager, offline and sync);
  5. DSA analysis with benchmarks;
  6. database design, integrity and indexing results;
  7. AI subsystems with evaluation and safety;
  8. testing;
  9. limitations and future work;
  10. an individual contribution statement signed by all five members.

Section 18

Academic Integrity and the Use of AI Tools

- **AI coding assistants are permitted, with disclosure.** `AI_USAGE.md` records which tools were used, what they were asked, what was accepted, and what was rejected and why. The last part carries the most weight, because judging AI output is the skill being assessed.
- **Everything you submit must be defensible in the viva.** A member who cannot explain, justify or modify any part of their own submission when asked loses the marks for it, whether or not it works. Undisclosed AI use is an integrity violation.
- **Study other systems; do not copy them.** Studying Puter, daedalOS and other open-source systems is encouraged. Copying their code, or another team's code, is prohibited.
- **Third-party code needs a licence and attribution.** Any third-party code or library must have a compatible licence and be listed in `THIRD_PARTY.md`. It may never stand in for a mandatory DSA structure (§10.2).
- **Secrets never go in the repository.** API keys, database credentials and tokens do not belong in the repository or the client bundle. A leaked key must be revoked at once and reported in `SECURITY.md`.
- **Contribution records must be authentic.** Fabricated commit history, contributions attributed to the wrong person, or an agreement to "carry" a member who does not contribute are treated as misconduct under university policy.

A final word

Few student projects touch as many layers of computer science at once as this one. Build it as engineers: measure before you claim, defend every choice, and make the system tell the truth, from the first frame of the boot screen to the last line of the audit log.

Give it a name you are proud of. It is your operating system.

Appendix A

Consolidated Acceptance Checklist

Run this checklist before every gate. Each line is verified live at the final defence on the reference machine. The full requirement text is in the section shown.

Area	Criterion	Target	Ref.	Done
Team	Registered team of exactly five students	5 members	**§4.1**	☐
Identity	Registered OS name, logo and version on boot, login, About and `uname`	present	**§5**	☐
Boot	Progress driven by real work; no timers in the boot path	0 timers	**BOOT-02**	☐
Boot	Warm boot / cold boot to an interactive desktop	≤ 1.5 s / ≤ 3.0 s	**BOOT-10**	☐
Boot	Cycle in a service manifest halts boot and names the cycle	panic screen	**BOOT-04**	☐
Boot	Boots to a usable desktop with the network disabled	yes	**BOOT-09**	☐

Area	Criterion	Target	Ref.	Done
Boot	Safe mode and recovery console reachable from the boot menu	both	**BOOT-06**	☐
Scheduler	p95 frame time with a CPU-bound process and 10 windows	≤ 16.7 ms	**PROC-05**	☐
Scheduler	Longest wait for a low-priority process under load	≤ declared bound	**PROC-04**	☐
Memory	Resident memory with a 2× budget workload	≤ budget	**MEM-06**	☐
Memory	Page-in of a 5 MB state, local / remote (p95)	≤ 300 ms / ≤ 2 s	**MEM-05**	☐
Memory	State after page-out and page-in	byte-identical	**MEM-03**	☐
VFS	Symlink loop detection	ELOOP at 40 hops	**VFS-03**	☐
VFS	Write beyond quota	EDQUOT, no partial data	**VFS-06**	☐
VFS	Directory nesting depth	≥ 32 levels	**VFS-08**	☐
VFS	Crashed lock holder's lease expires	within lease time	**VFS-09**	☐
Windows	p95 frame time while dragging with 10 windows open	≤ 16.7 ms	**WM-03**	☐
Offline	Core utilities offline; outbox replayed exactly once	0 lost, 0 duplicated	**OFF-01/02**	☐
Terminal	Trie autocomplete over ≥ 10,000 entries (p95)	≤ 1 ms	**SYS-03**	☐
Database	Fault-injected bulk operations: orphans after rollback	0	**DB-02**	☐
Database	Metadata queries Q1–Q5 at ≥ 10,000 records (p95)	≤ 10 ms	**DB-05**	☐
AI: RAG	Recall@5 on the evaluation set	≥ 0.80	**RAG-04**	☐
AI: RAG	Results the user is not allowed to read	0	**RAG-03**	☐
AI: RAG	Retrieval latency over ≥ 10,000 chunks (p95)	≤ 500 ms	**RAG-05**	☐
AI: Agent	Benchmark instructions completed correctly	≥ 16 of 20	**AGT-07**	☐
AI: Agent	Destructive actions run without confirmation	0, always	**AGT-03**	☐
AI: Agent	Tool calls triggered by injected file content	0	**AGT-06**	☐
AI: Pre-fetch	Top-3 accuracy against the best MFU/MRU baseline	higher	**PRE-02**	☐
AI: Pre-fetch	Median launch latency reduction on correct predictions	≥ 30%	**PRE-03**	☐
Delivery	Clean clone to a running system	≤ 3 commands, ≤ 5 min	**§17.1**	☐
Delivery	Line coverage overall / on dsa, vfs, kernel	≥ 70% / ≥ 85%	**§15.3**	☐

Appendix B
Glossary

Term	Meaning in this project
ACID	Atomicity, Consistency, Isolation, Durability: the guarantees a transaction gives. Here, above all, that a bulk operation happens completely or not at all.

Term	Meaning in this project
Aging	Gradually raising the priority of a process the longer it waits, so that low-priority work never starves.
Blast radius	The set of files and inodes an operation will create, modify or delete, shown to the user before an agent's plan runs.
Cosine similarity	(a·b)/(‖a‖‖b‖), the cosine of the angle between two vectors; 1 means the same direction, 0 means unrelated.
DAG	Directed acyclic graph. Here, the graph of service dependencies whose topological order is the boot order.
Embedding	A vector produced by a model to represent the meaning of a piece of text.
Idempotency key	A unique ID attached to an operation so that retrying it never applies it twice.
Inode	The record describing a file, directory or link: type, owner, mode, size, times and where its content is stored.
Kahn's algorithm	A topological sort that repeatedly removes nodes with no remaining incoming edges. If nodes remain at the end, the graph has a cycle.
Lease	A lock that expires unless renewed, so that a crashed client cannot hold a resource forever.
LRU	Least Recently Used: the replacement policy that evicts the entry that has gone unused the longest.
OPFS	Origin Private File System: a fast, private file-system storage area that the browser provides to each origin.
Outbox / operation log	A durable queue of local changes, replayed to the server in order when connectivity returns.
p95	The 95th percentile: 95% of measurements are at or below this value. Targets use p95 so that averages cannot hide slow cases.
Page fault	An access to state that is not resident in memory, which makes the memory manager page it in.
PCB	Process control block: the kernel's record for one process.
Prompt injection	An attack in which text the model reads, such as a file's contents, contains instructions meant to hijack its behaviour.
Quantum	The time slice a process may run before the scheduler reconsiders which process runs next.
RAG	Retrieval-augmented generation: retrieve relevant documents first, then generate an answer grounded in and citing them.
Recall@5 / MRR	Retrieval metrics: the fraction of queries whose relevant file appears in the top five, and the average of 1/rank of the first relevant result.
Service Worker	A browser script that intercepts network requests, making offline operation and cached shell updates possible.
Topological sort	An ordering of a DAG's nodes in which every node comes after all of the nodes it depends on.
Trie	A tree in which each edge is a character, so all words sharing a prefix share a path, which makes prefix completion fast.
Web Worker	A background thread in the browser that runs JavaScript off the main (UI) thread and talks to it by messages.

Appendix C

References and Further Reading

Benchmarks

- Puter, the open-source web desktop: `github.com/HeyPuter/puter`
- daedalOS, a desktop environment in the browser: `github.com/DustinBrett/daedalOS`

Operating systems

- R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, free online at `pages.cs.wisc.edu/~remzi/OSTEP/`. See especially the chapters on scheduling (MLFQ), paging and swapping policies, and file-system implementation.
- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, Wiley. Process management, CPU scheduling, virtual memory and file systems.

Data structures and algorithms

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press. Heaps, hash tables, B-trees, elementary graph algorithms and topological sort.

Databases

- PostgreSQL documentation: transaction isolation (`postgresql.org/docs/current/transaction-iso.html`) and index types (`postgresql.org/docs/current/indexes-types.html`).
- pgvector, vector similarity search for PostgreSQL: `github.com/pgvector/pgvector`

Browser platform

- MDN Web Docs: Web Workers API, IndexedDB API, File System API (Origin Private File System), Storage API, Service Worker API and WebSockets API. `developer.mozilla.org`
- web.dev: rendering performance, long tasks and offline-first patterns. `web.dev`

Artificial intelligence

- P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, NeurIPS 2020.
- N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, EMNLP 2019.
- Transformers.js, running models in the browser: `huggingface.co/docs/transformers.js`
- OWASP Top 10 for Large Language Model Applications, especially prompt injection and excessive agency: `owasp.org`

Appendix D

Team and OS Registration Form

Submit this form, completed and signed by all five members, by the end of **Week 2**. Commit a scanned copy to `docs/registration.pdf`. Names are allocated first come, first served.

Operating system identity

Proposed OS name _____

Alternative name (if the first is taken) _____

Tagline (one line) _____

Meaning or origin of the name _____

Repository URL _____ **Section** _____

Team members and roles (exactly five)

#	Name	Roll number	Primary role	Reviews	Signature
1			Kernel / Systems Architect		
2			VFS / Database Engineer		
3			Window Manager / UI Lead		
4			AI Pipeline Specialist		
5			QA / DevOps & Integration		

Intended AI subsystems (at least two)

☐ A. Semantic VFS search (RAG) ☐ B. Agentic system execution ☐ C. Predictive pre-fetching

Declaration

We confirm that our team has exactly five members; that the name above is original and appropriate under Section 5.1; that we have read this specification in full; that every member will be examined on the whole system; and that our marks will be moderated individually using the viva, the contribution evidence and peer assessment (Section 16.4). We have signed the team contract (Section 4.4).

Date _____ **Instructor approval** _____

Name registered as _____ **Team code** _____